

第 6 章 图

- 6.1 图的基本概念
- 6.2 图的存储结构
- 6.3 图的遍历
- 6.4 生成树和最小生成树
- 6.5 最短路径
- 6.6 拓扑排序
- 6.7 **AOE** 网与关键路径

本章小结

6.1 图的基本概念

6.1.1 图的定义

图 (Graph) G 由两个集合 $V(\text{Vertex})$ 和 $E(\text{Edge})$ 组成, 记为 $G=(V,E)$, 其中 V 是顶点的有限集合, 记为 $V(G)$, E 是连接 V 中两个不同顶点 (顶点对) 的边的有限集合, 记为 $E(G)$ 。

在图 G 中, 如果代表边的顶点对是无序的, 则称 G 为无向图, 无向图中代表边的无序顶点对通常用圆括号括起来, 用以表示一条无向边。

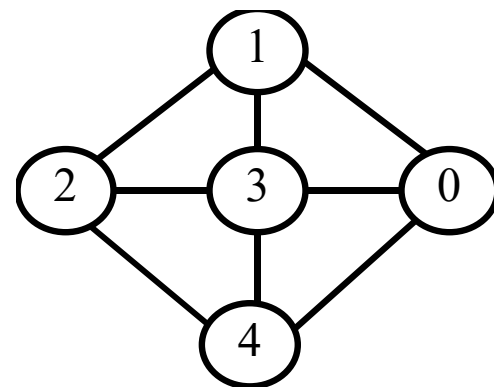
如果表示边的顶点对是有序的, 则称 G 为有向图, 在有向图中代表边的顶点对通常用尖括号括起来。

6.1.2 图的基本术语

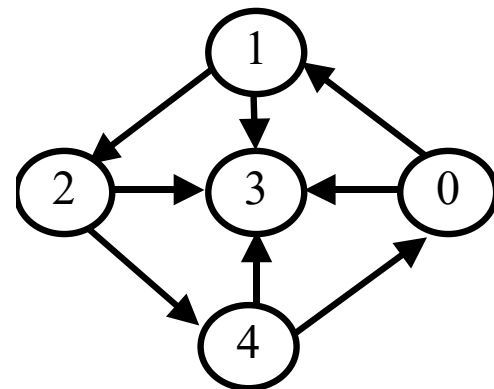
1. 端点和邻接点

在一个无向图中, 若存在一条边 (v_i, v_j) , 则称 v_i 和 v_j 为此边的两个端点, 并称它们互为邻接点。

在一个有向图中, 若存在一条边 $\langle v_i, v_j \rangle$, 则称此边是顶点 v_i 的一条出边, 同时也是顶点 v_j 的一条入边; 称 v_i 和 v_j 分别为此边的起始端点 (简称为起点)



(a)



(b)

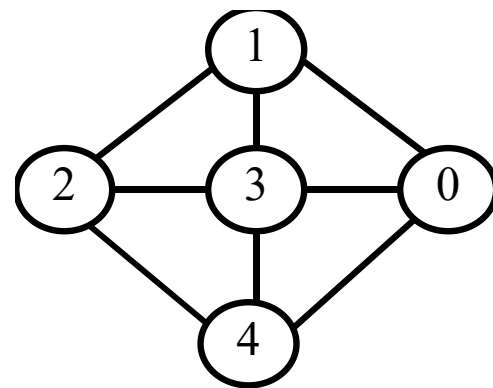
2. 顶点的度、入度和出度

在无向图中，顶点所具有的边的数目称为该顶点的度。

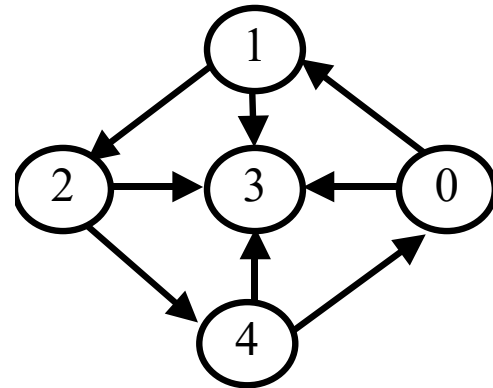
在有向图中，以顶点 v_i 为终点的入边的数目，称为该顶点的入度。以顶点 v_i 为始点的出边的数目，称为该顶点的出度。一个顶点的入度与出度的和为该顶点的度。

若一个图中有 n 个顶点和 e 条边，每个顶点的度为 $d_i (1 \leq i \leq n)$ ，则有：

$$e = \frac{1}{2} \sum_{i=1}^n d_i$$



(a)

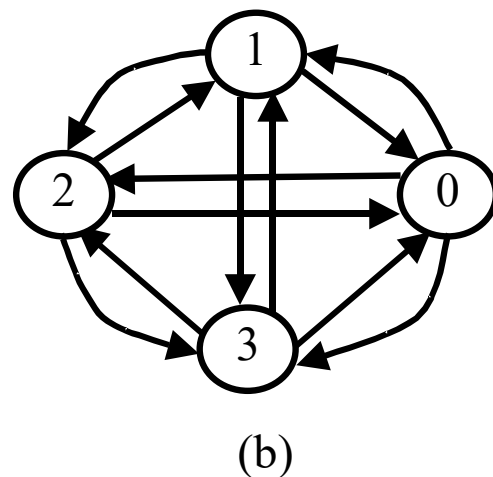
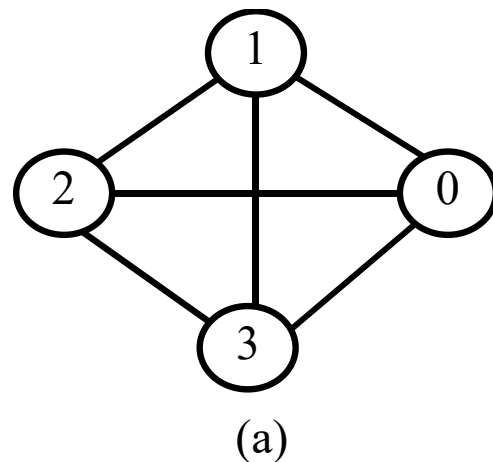


(b)

3. 完全图

若无向图中的每两个顶点之间都存在着一边，有向图中的每两个顶点之间都存在着方向相反的两条边，则称此图为完全图。

显然，完全无向图包含有 $n(n-1)/2$ 条边，完全有向图包含有 $n(n-1)$ 条边。例如，图 (a) 所示的图是一个具有 4 个顶点的完全无向图，共有 6 条边。图 (b) 所示的图是一个具有 4 个顶点的完全有向图，共有 12 条边。



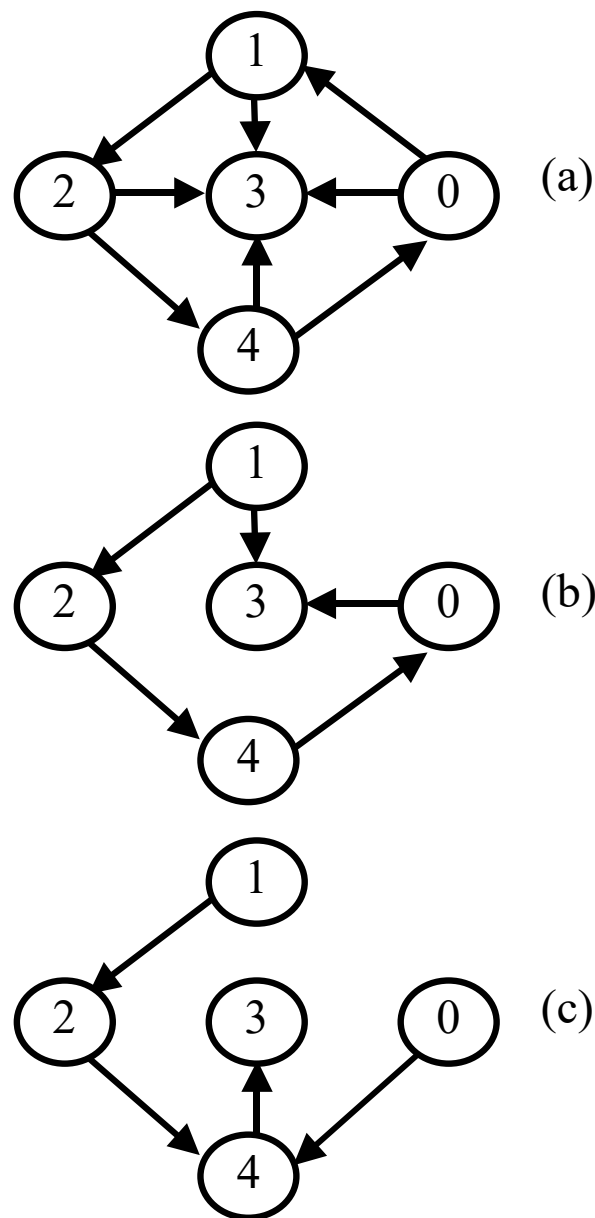
4. 稠密图、稀疏图

当一个图接近完全图时，则称为稠密图。相反，当一个图含有较少的边数（即当 $e \ll n(n-1)$ ）时，则称为稀疏图。

5. 子图

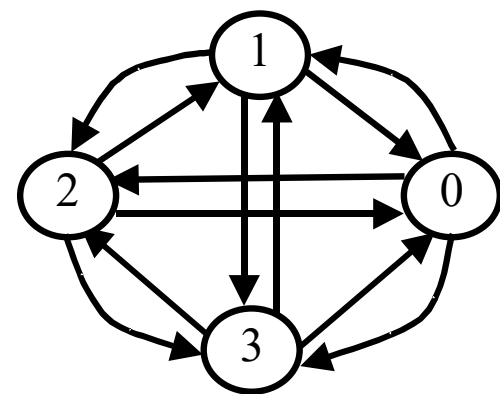
设有两个图 $G=(V,E)$ 和 $G'=(V',E')$ ，若 V' 是 V 的子集，即 $V' \subseteq V$ ，且 E' 是 E 的子集，即 $E' \subseteq E$ ，则称 G' 是 G 的子图。例如图 (b) 是图 (a) 的子图，而图 (c) 不是图 (a) 的子图。

精选课件



6. 路径和路径长度

在一个图 $G=(V,E)$ 中, 从顶点 v_i 到顶点 v_j 的一条路径是一个顶点序列 $(v_i, v_{i1}, v_{i2}, \dots, v_{im}, v_j)$, 若此图 G 是无向图, 则边 $(v_i, v_{i1}), (v_{i1}, v_{i2}), \dots, (v_{im-1}, v_{im}), (v_{im}, v_j)$ 属于 $E(G)$; 若此图是有向图, 则 $\langle v_i, v_{i1} \rangle, \langle v_{i1}, v_{i2} \rangle, \dots, \langle v_{im-1}, v_{im} \rangle, \langle v_{im}, v_j \rangle$ 属于 $E(G)$ 。

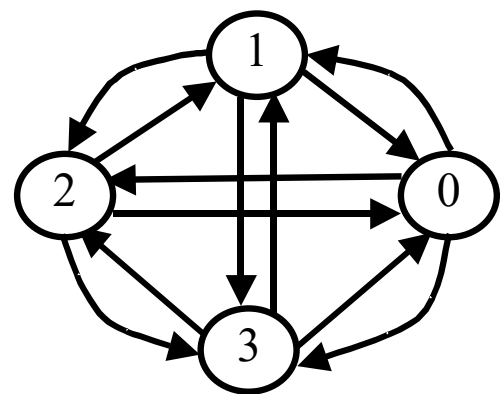


路径长度是指一条路径上经过的边的数目。若一条路径上除开始点和结束点可以相同外, 其余顶点均不相同, 则称此路径为简单路径。例如, 有图中,

精确课件

7. 回路或环

若一条路径上的开始点与结束点为同一个顶点，则此路径被称为回路或环。开始点与结束点相同的简单路径被称为简单回路或简单环。



例如，右图中 (v_0, v_2, v_1, v_0) 就是一条简单回路，其长度为 3。

8. 连通、连通图和连通分量

在无向图 G 中, 若从顶点 v_i 到顶点 v_j 有路径, 则称 v_i 和 v_j 是连通的。

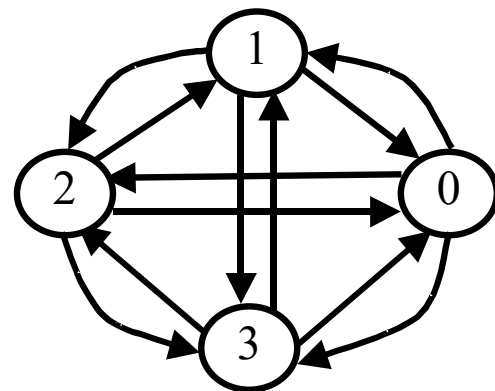
若图 G 中任意两个顶点都连通, 则称 G 为连通图, 否则称为非连通图。

无向图 G 中的极大连通子图称为 G 的连通分量。显然, 任何连通图的连通分量只有一个, 即本身, 而非连通图有多个连通分量。

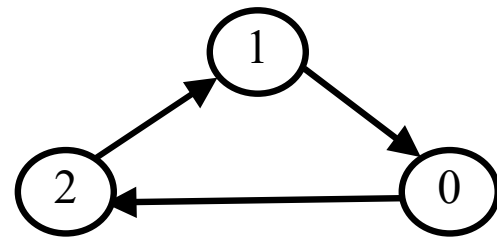
6. 强连通图和强连通分量

在有向图 G 中, 若从顶点 v_i 到顶点 v_j 有路径, 则称从 v_i 到 v_j 是连通的。

若图 G 中的任意两个顶点 v_i 和 v_j 都连通, 即从 v_i 到 v_j 和从 v_j 到 v_i 都存在路径, 则称图 G 是强连通图。例如, 右边两个图都是强连通图。



(a)



(b)

有向图 G 中的极大强连通子图称为 G 的强连通分量。显然

10. 权和网

图中每一条边都可以附有一个对应的数值，这种与边相关的数值称为权。权可以表示从一个顶点到另一个顶点的距离或花费的代价。边上带有权的图称为带权图，也称作网。

例 6.1 有 n 个顶点的有向强连通图最多需要多少条边？最少需要多少条边？

答：有 n 个顶点的有向强连通图最多有 $n(n-1)$ 条边（构成一个有向完全图的情况）；最少有 n 条边（ n 个顶点依次首尾相接构成一个环的情况）。

6.2 图的存储结构

6.2.1 邻接矩阵存储方法

邻接矩阵是表示顶点之间相邻关系的矩阵。
设 $G=(V,E)$ 是具有 $n(n > 0)$ 个顶点的图, 顶点的顺序依次为 $(v_0, v_1, \dots, v_{n-1})$, 则 G 的邻接矩阵 A 是 n 阶方阵, 其定义如下:

(1) 如果 G 是无向图, 则:

$A[i][j] = 1$: 若 $(v_i, v_j) \in E(G)$ 0 : 其他

(2) 如果 G 是有向图, 则:

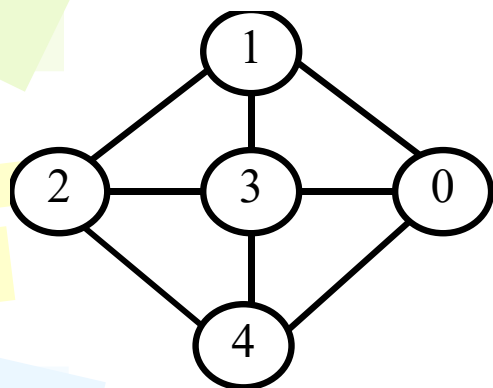
$A[i][j] = 1$: 若 $\langle v_i, v_j \rangle \in E(G)$ 0 : 其他

(3) 如果 G 是带权无向图, 则:

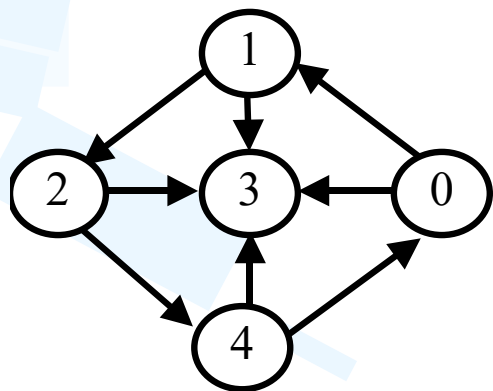
$A[i][j] = w_{ij}$: 若 $v_i \neq v_j$ 且 $(v_i, v_j) \in E(G)$ ∞ : 其他

(4) 如果 G 是带权有向图, 则:

$A[i][j] = w_{ij}$: 若 $v_i \neq v_j$ 且 $\langle v_i, v_j \rangle \in E(G)$ ∞ : 其他



(a)



(b)

$$A1 = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{bmatrix}$$

$$A2 = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

邻接矩阵的特点如下：

(1) 图的邻接矩阵表示是惟一的。

(2) 无向图的邻接矩阵一定是一个对称矩阵。
因此，按照压缩存储的思想，在具体存放邻接矩阵时只需存放上（或下）三角形阵的元素即可。

(3) 不带权的有向图的邻接矩阵一般来说是一个稀疏矩阵，因此，当图的顶点较多时，可以采用三元组表的方法存储邻接矩阵。

(4) 对于无向图，邻接矩阵的第 i 行（或第 i 列）非零元素（或非 ∞ 元素）的个数正好是第 i 个顶点 v_i 的度。

(5) 对于有向图，邻接矩阵的第 i 行（或第 i 列）非零元素（或非 ∞ 元素）的个数正好是第 i 个顶点 v_i 的出度（或入度）。

(6) 用邻接矩阵方法存储图，很容易确定图中任意两个顶点之间是否有边相连。但是，要确定图中有多少条边，则必须按行、按列对每个元素进行检测，所花费的时间代价很大。这是用邻接矩阵存储图的局限性。

邻接矩阵的数据类型定义如下:

```
#define MAXV <最大顶点个数>
```

```
typedef struct
```

```
{    int no;
```

```
/* 顶点编号 */
```

```
    InfoType info;
```

```
/* 顶点其他信息 */
```

```
} VertexType;
```

```
/* 顶点类型 */
```

```
typedef struct
```

```
/* 图的定义 */
```

```
{    int edges[MAXV][MAXV];
```

```
/* 邻接矩阵 */
```

```
    int vexnum,arcnum;
```

```
/* 顶点数 , 弧数 */
```

```
    VertexType veks[MAXV];
```

```
/* 存放顶点信息 */
```

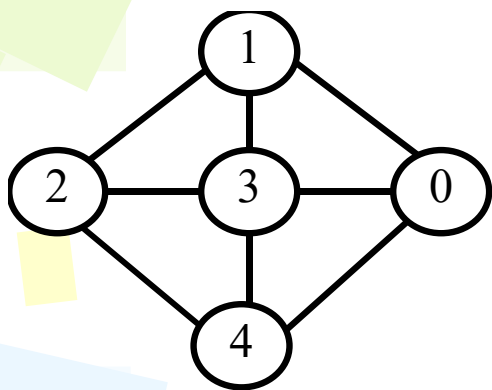
```
} MGraph;
```

6.2.2 邻接表存储方法

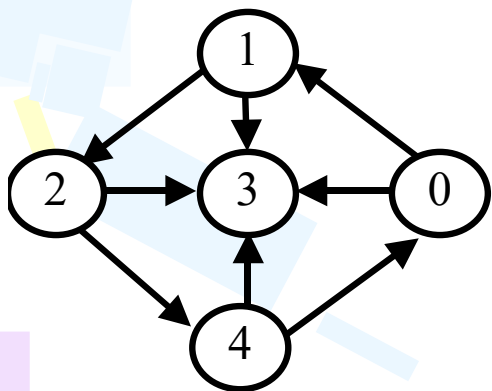
图的邻接表存储方法是一种顺序分配与链式分配相结合的存储方法。在邻接表中,对图中每个顶点建立一个单链表,第 i 个单链表中的结点表示依附于顶点 v_i 的边(对有向图是以顶点 v_i 为尾的弧)。每个单链表上附设一个表头结点。表结点和表头结点的结构如下:



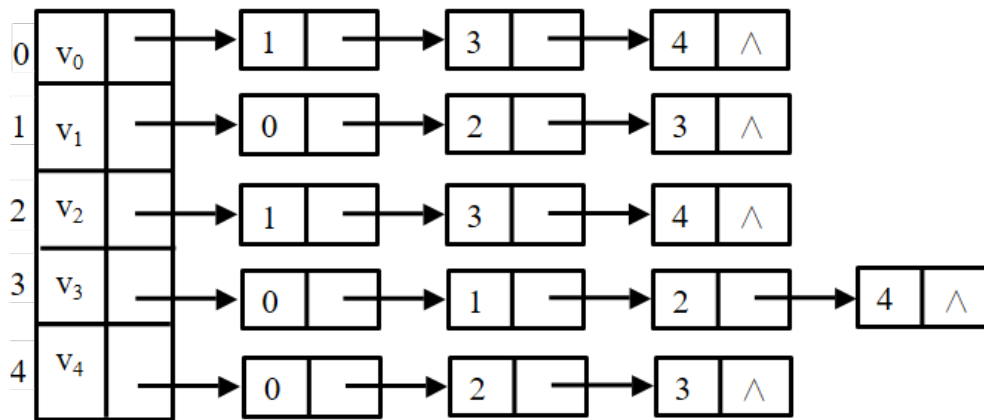
其中,表结点由三个域组成,adjvex 指示与顶点 v_i 邻接的点在图中的位置,nextarc 指示下一条边或弧的结点,info 存储与边或弧相关的信息,如权值等。表头结点由两个域组成,data 存储顶点 v_i 的名称或其他信息,firstarc 指向链表中第一个结点。



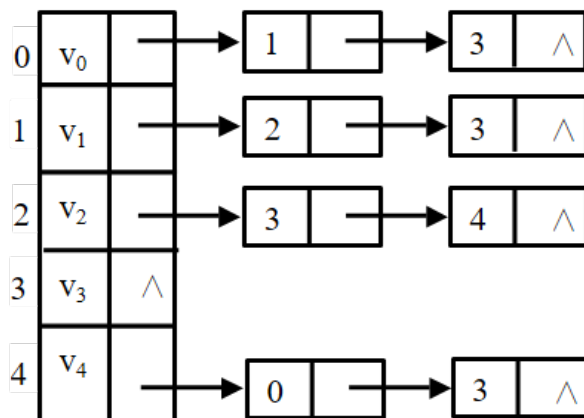
(a)



(b)



(a)



(b)

邻接表的特点如下：

(1) 邻接表表示不惟一。这是因为在每个顶点对应的单链表中，各边结点的链接次序可以是任意的，取决于建立邻接表的算法以及边的输入次序。

(2) 对于有 n 个顶点和 e 条边的无向图，其邻接表有 n 个顶点结点和 $2e$ 个边结点。显然，在总的边数小于 $n(n-1)/2$ 的情况下，邻接表比邻接矩阵要节省空间。

(3) 对于无向图，邻接表的顶点 v_i 对应的第 i 个链表的边结点数目正好是顶点 v_i 的度。

(4) 对于有向图，邻接表的顶点 v_i 对应的第 i 个链表的边结点数目仅仅是 v_i 的出度。其入度为邻接表中所有 `adjvex` 域值为 i 的边结点数目。

邻接表存储结构的定义如下：

```
typedef struct ANode          /* 弧的结点结构类型 */
{
    int adjvex;               /* 该弧的终点位置 */
    struct ANode *nextarc;    /* 指向下一条弧的指针 */
    InfoType info;           /* 该弧的相关信息 */
} ArcNode;

typedef struct Vnode          /* 邻接表头结点的类型 */
{
    Vertex data;              /* 顶点信息 */
    ArcNode *firstarc;        /* 指向第一条弧 */
} VNode;

typedef VNode AdjList[MAXV]; /* AdjList 是邻接表类型 */

typedef struct
{
    AdjList adjlist;          /* 邻接表 */
    int n,e;                  /* 图中顶点数 n 和边数 e */
} ALGraph;                   /* 图的类型 */
```


例 6.2 给定一个具有 n 个结点的无向图的邻接矩阵和邻接表。

- (1) 设计一个将邻接矩阵转换为邻接表的算法;
- (2) 设计一个将邻接表转换为邻接矩阵的算法;
- (3) 分析上述两个算法的时间复杂度。

解：(1) 在邻接矩阵上查找值不为 0 的元素，找到这样的元素后创建一个表结点并在邻接表对应的单链表中采用前插法插入该结点。算法如下：

```
void MatToList(MGraph g,ALGraph *&G)
```

```
/* 将邻接矩阵 g 转换成邻接表 G*/
```

```
{ int i,j,n=g.vexnum; ArcNode *p;          /*n 为顶点数 */
```

```
G=(ALGraph *)malloc(sizeof(ALGraph));
```

```
for (i=0;i<n;i++) /* 给所有头结点的指针域置初值 */
```

```
G->adjlist[i].firstarc=NULL;
```

```
for (i=0;i<n;i++) /* 检查邻接矩阵中每个元素 */
```

```
for (j=n-1;j>=0;j--)
```

```
if (g.edges[i][j]!=0)
```

```
{ p=(ArcNode *)malloc(sizeof(ArcNode)); /* 创建结点
```

```
*p*/
```

```
p->adjvex=j;
```

```
p->nextarc=G->adjlist[i].firstarc; /* 将 *p 链到链表后 */
```

```
G->adjlist[i].firstarc=p;
```

```
}
```

```
G->n=n;G->e=g.arcnum;
```

(2) 在邻接表上查找相邻结点，找到后修改相应邻接矩阵元素的值。算法如下：

```
void ListToMat(ALGraph *G,MGraph &g)
{
    int i,j,n=G->n;ArcNode *p;
    for (i=0;i<n;i++)
    {
        p=G->adjlist[i].firstarc;
        while (p!=NULL)
        {
            g.edges[i][p->adjvex]=1;
            p=p->nextarc;
        }
    }
    g.vexnum=n;g.arcnum=G->e;
}
```



(3) 算法 1 的时间复杂度均为 $O(n^2)$ 。算法 2 的时间复杂度为 $O(n+e)$, 其中 e 为图的边数。

6.3 图的遍历

6.3.1 图的遍历的概念

从给定图中任意指定的顶点（称为初始点）出发，按照某种搜索方法沿着图的边访问图中的所有顶点，使每个顶点仅被访问一次，这个过程称为图的遍历。如果给定图是连通的无向图或者是强连通的有向图，则遍历过程一次就能完成，并可按访问的先后顺序得到由该图所有顶点组成的一个序列。

根据搜索方法的不同，图的遍历方法有两种：一种叫做深度优先搜索法 (DFS)；另一种叫做广度优先搜索法 (BFS)。

6.3.2 深度优先搜索遍历

深度优先搜索遍历的过程是：从图中某个初始顶点 v 出发，首先访问初始顶点 v ，然后选择一个与顶点 v 相邻且没被访问过的顶点 w 为初始顶点，再从 w 出发进行深度优先搜索，直到图中与当前顶点 v 邻接的所有顶点都被访问过为止。显然，这个遍历过程是个递归过程。

以邻接表为存储结构的深度优先搜索遍历算法如下（其中， v 是初始顶点编号， $visited[]$ 是一个全局数组，初始时所有元素均为 0 表示所有顶点尚未访问过）：

```
void DFS(ALGraph *G,int v)
```

```
{   ArcNode *p;
```

```
    visited[v]=1;
```

```
    /* 置已访问标记 */
```

```
    printf("%d ",v);
```

```
    /* 输出被访问顶点的编号 */
```

```
    p=G->adjlist[v].firstarc;
```

```
    /*p 指向顶点 v 的第一条弧的弧头结点 */
```

```
    while (p!=NULL)
```

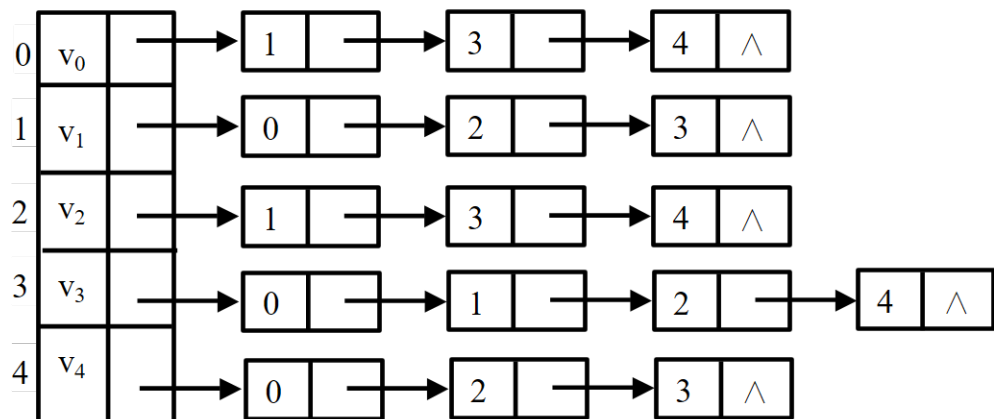
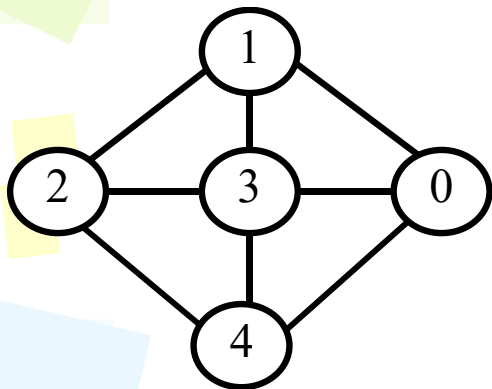
```
    {   if (visited[p->adjvex]==0) DFS(G,p->adjvex);
```

```
        /* 若 p->adjvex 顶点未访问 , 递归访问它 */
```

```
        p=p->nextarc;
```

```
        /*p 指向顶点 v 的下一条弧的弧头结点 */
```

```
    }
```



例如，以上图的邻接表为例调用 **DFS()** 函数，假设初始顶点编号 $v=2$ ，给出调用 **DFS()** 的执行过程，见教材。

6.3.3 广度优先搜索遍历

广度优先搜索遍历的过程是：首先访问初始点 v_i ，接着访问 v_i 的所有未被访问过的邻接点 $v_{i1}, v_{i2}, \dots, v_{it}$ ，然后再按照 $v_{i1}, v_{i2}, \dots, v_{it}$ 的次序，访问每一个顶点的所有未被访问过的邻接点，依次类推，直到图中所有和初始点 v_i 有路径相通的顶点都被访问过为止。

以邻接表为存储结构，用广度优先搜索遍历图时，需要使用一个队列，以类似于按层次遍历二叉

```
void BFS(ALGraph *G,int v)
```

```
{   ArcNode *p; int w,i;
```

```
int queue[MAXV],front=0,rear=0;   /* 定义循环队列 */
```

```
int visited[MAXV]; /* 定义存放结点的访问标志的数组 */
```

```
for (i=0;i<G->n;i++) visited[i]=0; /* 访问标志数组初始化 */
```

```
printf("%2d",v);           /* 输出被访问顶点的编号 */
```

```
visited[v]=1;              /* 置已访问标记 */
```

```
rear=(rear+1)%MAXV;
```

```
queue[rear]=v;             /*v 进队 */
```

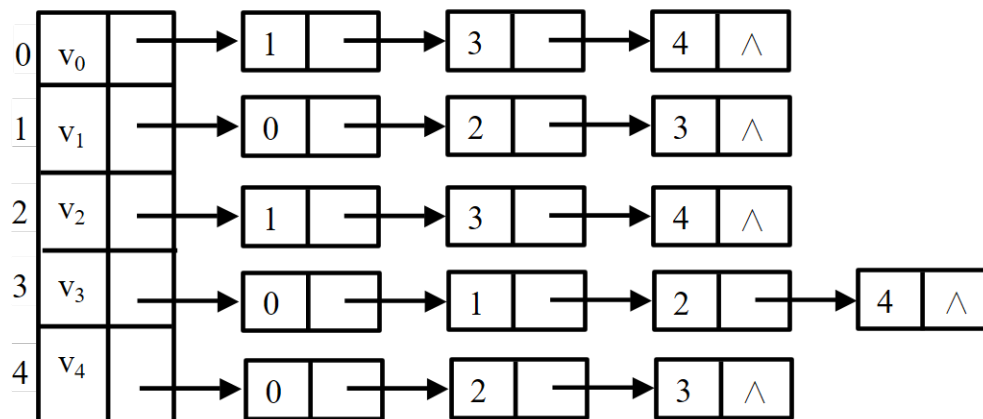
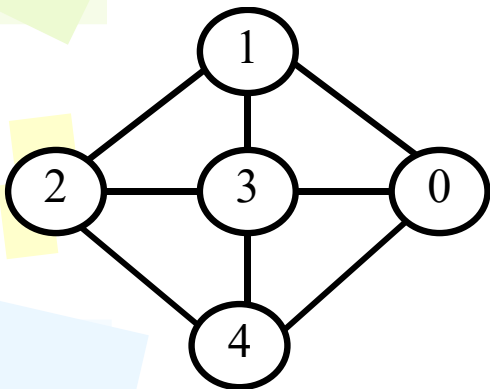
```

while (front!=rear)                /* 若队列不空时循环 */
{
    front=(front+1)%MAXV;
    w=queue[front];                /* 出队并赋给 w */
    p=G->adjlist[w].firstarc; /* 找 w 的第一个的邻接点 */

    while (p!=NULL)
    {
        if (visited[p->adjvex]==0)
        {
            printf("%2d",p->adjvex); /* 访问之 */
            visited[p->adjvex]=1;
            rear=(rear+1)%MAXV; /* 该顶点进队 */
            queue[rear]=p->adjvex;
        }
        p=p->nextarc; /* 找下一个邻接顶点 */
    }
}

printf("\n");
}

```

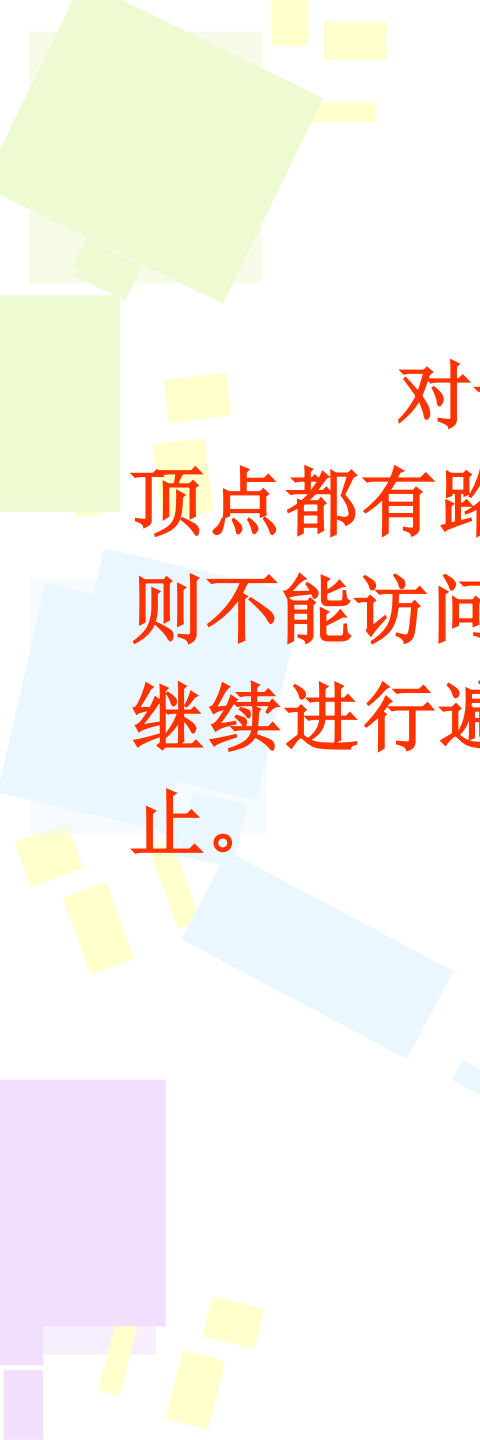


例如，以上图的邻接表为例调用 **BFS()** 函数，假设初始顶点编号 $v=2$ ，给出调用 **BFS()** 的执行过程，见教材。

6.3.4 非连通图的遍历

对于无向图来说，若无向图是连通图，则一次遍历能够访问到图中的所有顶点；

但若无向图是非连通图，则只能访问到初始点所在连通分量中的所有顶点，其他连通分量中的顶点是不可能访问到的。为此需要从其他每个连通分量中选择初始点，分别进行遍历，才能够访问到图中的所有顶点；



对于有向图来说，若从初始点到图中的每个顶点都有路径，则能够访问到图中的所有顶点；否则不能访问到所有顶点，为此同样需要再选初始点，继续进行遍历，直到图中的所有顶点都被访问过为止。

采用深度优先搜索遍历非连通无向图的算法如下：

```
DFS1(ALGraph *g)
```

```
{
```

```
    int i;
```

```
    for (i=0;i<g->n;i+)    /* 遍历所有未访问过的顶点 */
```

```
        if (visited[i]==0)
```

```
            DFS(g,i);
```

```
}
```

采用广度优先搜索遍历非连通无向图的算法如下：

```
BFS1(ALGraph *g)
```

```
{  
    int i;  
    for (i=0;i<g->n;i+) /* 遍历所有未访问过的顶点 */  
        if (visited[i]==0)  
            BFS(g,i);  
}
```


6.3.5 图遍历的应用

例 假设图 **G** 采用邻接表存储，设计一个算法，判断无向图 **G** 是否连通。若连通则返回 **1**；否则返回 **0**。

解：采用遍历方式判断无向图 **G** 是否连通。这里用深度优先遍历方法，先给 `visited[]` 数组（为全局变量）置初值 **0**，然后从 **0** 顶点开始遍历该图。

在一次遍历之后，若所有顶点 *i* 的 `visited[i]` 均为 **1**，则该图是连通的；否则不连通。对应的算法如下：

```

int Connect(ALGraph *G)          /* 判断无向图 G 的连通性
*/
{
    int i,flag=1;
    for (i=0;i<G->n;i++) /*visited 数组置初值 */
        visited[i]=0;
    DFS(G,0);/* 调用 DSF 算法 , 从顶点 0 开始深度优先遍
历 */
    for (i=0;i<G->n;i++)
        if (visited[i]==0)
        {
            flag=0;
            break;
        }
    return flag;
}

```

例 假设图 G 采用邻接表存储，设计一个算法，输出图 G 中从顶点 u 到 v 的长度为 l 的所有简单路径。

解：所谓简单路径是指路径上的顶点不重复。利用回溯的深度优先搜索方法。

从顶点 u 开始，进行深度优先搜索，在搜索过程中，需要把当前的搜索线路记录下来。为此设立一个数组 $path$ 保存走过的路径，用 d 记录走过的路径长度。若当前扫描到的结点 u 等于 v 且路径长度为 l 时，表示找到了一条路径，则输出路径 $path$ 。

对应的算法如下：

```

void PathAll(ALGraph *G,int u,int v,int l,int path[],int d)
/*d 是到当前为止已走过的路径长度，调用时初值为 -1*/
{
    int m,i; ArcNode *p; visited[u]=1; d++; /* 路径长度增 1*/
    path[d]=u; /* 将当前顶点添加到路径中 */
    if (u==v && d==l) /* 输出一条路径 */
    {
        printf(" ");
        for (i=0;i<=d;i++) printf("%d ",path[i]);
        printf("\n"); }
    p=G->adjlist[u].firstarc; /*p 指向 u 的第一条弧的弧头结点
*/
    while (p!=NULL)
    {
        m=p->adjvex; /*m 为 u 的邻接顶点 */
        if (visited[m]==0) /* 若顶点未标记访问，则递归访问之
*/
            PathAll(G,m,v,l,path,d);
        p=p->nextarc /* 找 u 的下一个邻接顶点 */
    }
    visited[u]=0; /* 恢复环境 */
}

```

```

void main()
{
    int path[MAXV],u=1,v=4,d=3,i,j;
    MGraph g;
    ALGraph *G;
    g.n=5;g.e=6;
    int A[MAXV][MAXV]={ {0,1,0,1,0},{1,0,1,0,0},
                        {0,1,0,1,1}, {1,0,1,0,1}, {0,0,1,1,0} };
    for (i=0;i<g.n;i++)    /* 建立图 6.1(a) 的邻接矩阵 */
        for (j=0;j<g.n;j++)
            g.edges[i][j]=A[i][j];
    MatToList(g,G);
    for (i=0;i<g.n;i++) visited[i]=0;
    printf(" 图 G:\n");DispAdj(G);/* 输出邻接表 */
    printf(" 从 %d 到 %d 的所有长度为 %d 路径 :
\n",u,v,d);
    PathAll(G,u,v,d,path,-1);
    printf("\n");
}

```

程序执行结果如下：

图 G:

0: 1 3

1: 0 2

2: 1 3 4

3: 0 2 4

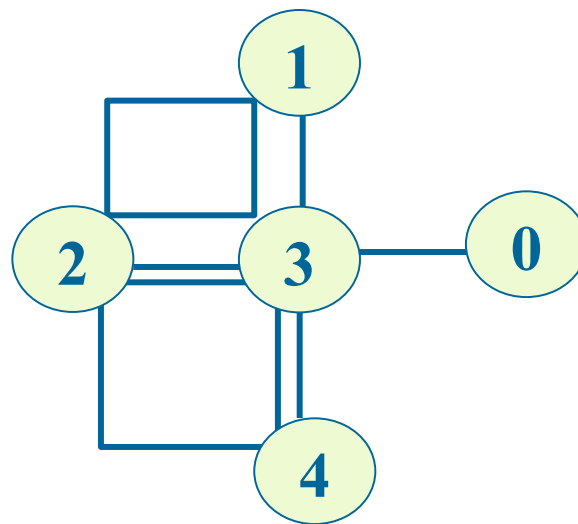
4: 2 3

从 1 到 4 的所有长度为 3 路

径：

1 0 3 4

1 2 3 4



6.4 生成树和最小生成树

6.4.1 生成树的概念

一个连通图的生成树是一个极小连通子图，它含有图中全部顶点，但只有构成一棵树的 $(n-1)$ 条边。

如果在一棵生成树上添加一条边，必定构成一个环：因为这条边使得它依附的那两个顶点之间有了第二条路径。一棵有 n 个顶点的生成树（连通无回路图）有且仅有 $(n-1)$ 条边，如果一个图有 n 个顶点和小于 $(n-1)$ 条边，则是非连通图。如果它多于 $(n-1)$ 条边，则一定有回路。但是，有 $(n-1)$ 条边的图不一定是生成树。

对于一个带权（假定每条边上的权均为大于零的实数）连通无向图 G 中的不同生成树，其每棵树上所有边上的权值之和也可能不同；图的所有生成树中具有边上的权值之和最小的树称为图的最小生成树。

按照生成树的定义， n 个顶点的连通图的生成树有 n 个顶点、 $n-1$ 条边。因此，构造最小生成树的准则有三条：

(1) 必须只使用该图中的边来构造最小生成树；

(2) 必须使用且仅使用 $n-1$ 条边来连接图中的 n 个顶点；

(3) 不能使用产生回路的边。

6.4.2 无向图的连通分量和生成树

在对无向图进行遍历时，对于连通图，仅需调用遍历过程 (DFS 或 BFS) 一次，从图中任一顶点出发，便可以遍历图中的各个顶点。对非连通图，则需多次调用遍历过程，每次调用得到的顶点集连同相关的边就构成图的一个连通分量。

设 $G=(V,E)$ 为连通图，则从图中任一顶点出发遍历图时，必定将 $E(G)$ 分成两个集合 T 和 B ，其中 T 是遍历图过程中走过的边的集合， B 是剩余的边的集合： $T \cap B = \Phi, T \cup B = E(G)$ 。显然， $G'=(V,T)$ 是 G 的极小连通子图，即 G' 是 G 的一棵生成树。

由深度优先遍历得到的生成树称为深度优先生成树；由广度优先遍历得到的生成树称为广度优先生成树。这样的生成树是由遍历时访问过的 n 个顶点和遍历时经历的 $n-1$ 条边组成。

对于非连通图，每个连通分量中的顶点集和遍历时走过的边一起构成一棵生成树，各个连通分量的生成树组成非连通图的生成森林。

6.4.4 普里姆算法

普里姆 (Prim) 算法是一种构造性算法。

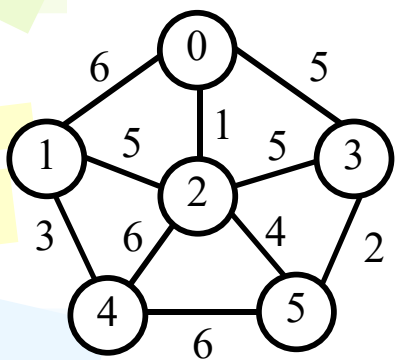
假设 $G=(V,E)$ 是一个具有 n 个顶点的带权连通无向图, $T=(U,TE)$ 是 G 的最小生成树, 其中 U 是 T 的顶点集, TE 是 T 的边集, 则由 G 构造最小生成树 T 的步骤如下:

(1) 初始化 $U=\{v_0\}$ 。 v_0 到其他顶点的所有边为候选边；

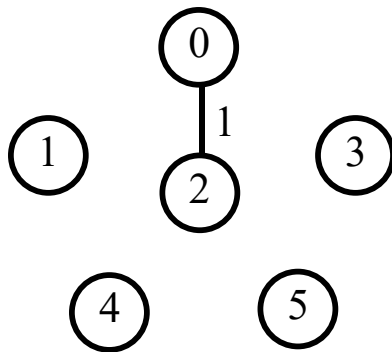
(2) 重复以下步骤 $n-1$ 次, 使得其他 $n-1$ 个顶点被加入到 U 中:

① 从候选边中挑选权值最小的边输出, 设该边在 $V-U$ 中的顶点是 v , 将 v 加入 U 中, 删除和 v 关联的边;

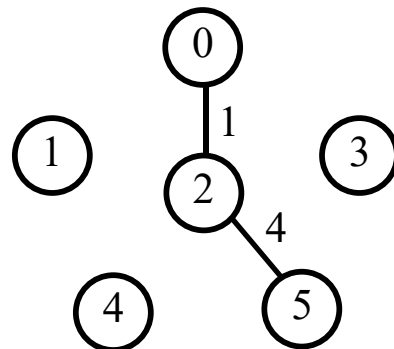
② 考察当前 $V-U$ 中的所有顶点 v_i , 修改候选边: 若 (v, v_i) 的权值小于原来和 v_i 关联的候选边, 则用 (v, v_i) 取代后者作为候选边。



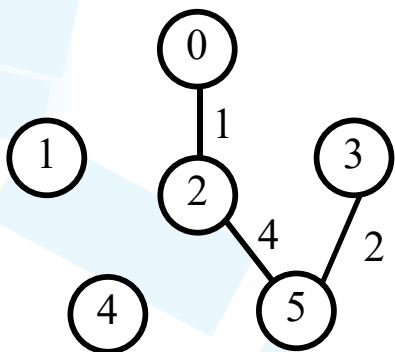
(a)



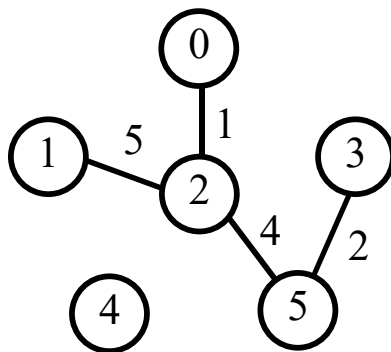
(b)



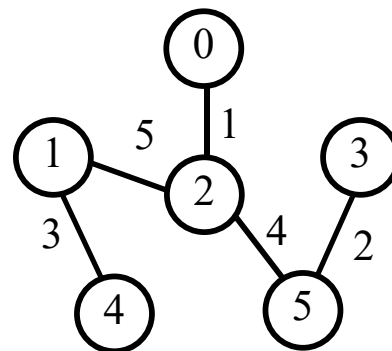
(c)



(d)



(e)



(f)

普里姆算法求解最小生成树的过程

精选课件

普里姆 (Prim) 算法如下:

```
#define INF 32767    /*INF 表示 $\infty$  */
```

```
void Prim(int cost[][MAXV],int n,int v)
```

```
{    int lowcost[MAXV],min;
```

```
    int closest[MAXV],i,j,k;
```

```
    for (i=0;i<n;i++)    /* 给 lowcost[] 和 closest[] 置初值 */
```

```
{
```

```
        lowcost[i]=cost[v][i];
```

```
        closest[i]=v;
```

```
}
```

```
for (i=1;i<n;i++)      /* 找出 n-1 个顶点 */
```

```
{  min=INF;
```

```
    for (j=0;j<n;j++) /* 在 (V-U) 中找出离 U 最近的顶点
```

```
    k*/
```

```
        if (lowcost[j]!=0 && lowcost[j]<min)
```

```
        {  min=lowcost[j]; k=j;  }
```

```
    printf(" 边 (%d,%d) 权为 :%d\n",closest[k],k,min);
```

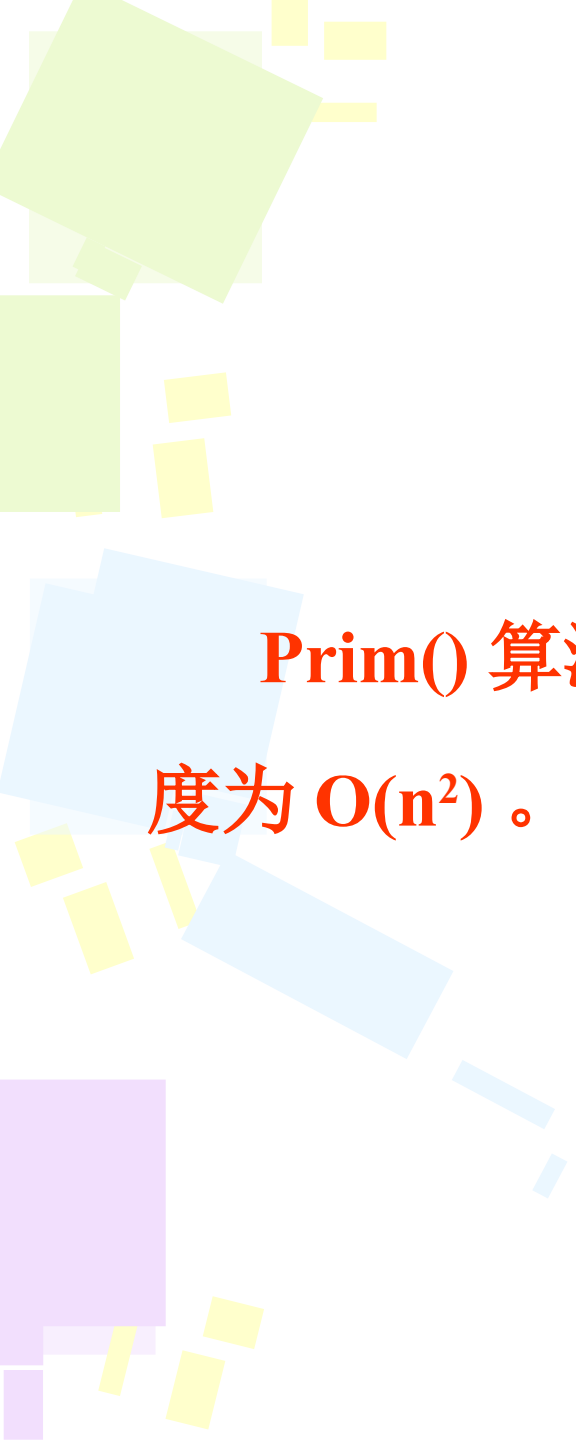
```
    lowcost[k]=0;          /* 标记 k 已经加入 U*/
```

```
    for (j=0;j<n;j++)      /* 修改数组 lowcost 和
```

```
    closest*/
```

```
        if (cost[k][j]!=0 && cost[k][j]<lowcost[j])
```

```
        {  lowcost[j]=cost[k][j]; closest[j]=k; }
```



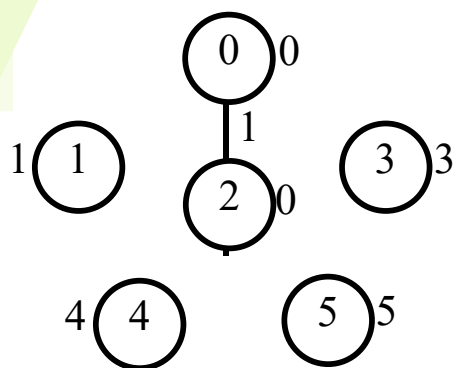
Prim() 算法中有两重 for 循环 , 所以时间复杂度为 $O(n^2)$ 。

6.4.5 克鲁斯卡尔算法

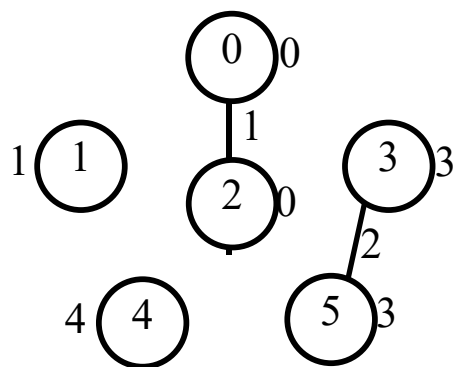
克鲁斯卡尔 (Kruskal) 算法是一种按权值的递增次序选择合适的边来构造最小生成树的方法。假设 $G=(V,E)$ 是一个具有 n 个顶点的带权连通无向图, $T=(U,TE)$ 是 G 的最小生成树, 则构造最小生成树的步骤如下:

(1) 置 U 的初值等于 V (即包含有 G 中的全部顶点), TE 的初值为空集 (即图 T 中每一个顶点都构成一个分量)。

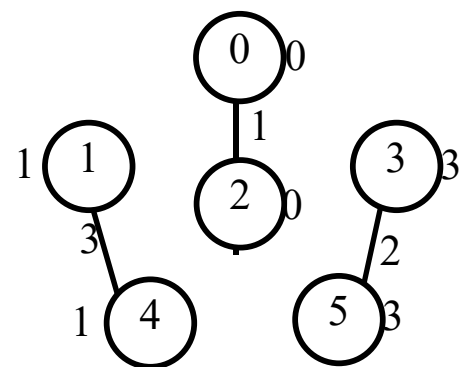
(2) 将图 G 中的边按权值从小到大的顺序依次选取: 若选取的边未使生成树 T 形成回路, 则加入 TE ; 否则



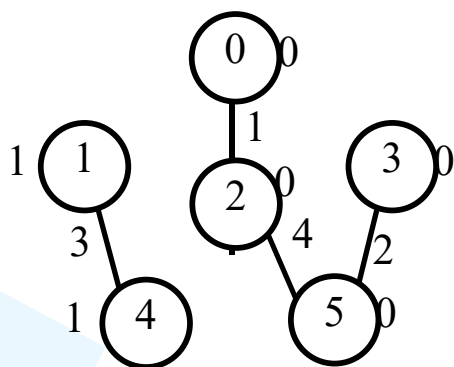
(a)



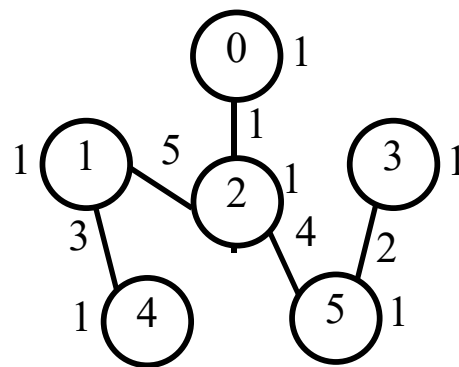
(b)



(c)



(d)



(e)

克鲁斯卡尔算法求解最小生成树的过程

为了简便，在实现克鲁斯卡尔算法 `Kruskal()` 时，参数 `E` 存放图 `G` 中的所有边，假设它们是按权值从小到大的顺序排列的。`n` 为图 `G` 的顶点个数，`e` 为图 `G` 的边数。

```
typedef struct
{
    int u;    /* 边的起始顶点 */
    int v;    /* 边的终止顶点 */
    int w;    /* 边的权值 */
} Edge;
```

```
void Kruskal(Edge E[],int n,int e)
```

```
{  int i,j,m1,m2,sn1,sn2,k; int vset[MAXV];
```

```
    for (i=0;i<n;i++) vset[i]=i;      /* 初始化辅助数组 */
```

```
    k=1; /*k 表示当前构造最小生成树的第几条边,初值为 1*/
```

```
    j=0; /*E 中边的下标,初值为 0*/
```

```
    while (k<n) /* 生成的边数小于 n 时循环 */
```

```
{      m1=E[j].u;m2=E[j].v;      /* 取一条边的头尾顶点 */
```

```
        sn1=vset[m1];sn2=vset[m2];
```

```
        /* 分别得到两个顶点所属的集合编号 */
```

```
if (sn1!=sn2)  /* 属不同的集合 -> 最小生成树的一条边 */
{
    printf("(%d,%d):%d\n",m1,m2,E[j].w);
    k++;          /* 生成边数增 1*/
    for (i=0;i<n;i++)  /* 两个集合统一编号 */
        if (vset[i]==sn2) /* 集合编号为 sn2 的改为 sn1*/
            vset[i]=sn1;
    }
    j++;          /* 扫描下一条边 */
}
```

完整的克鲁斯卡尔算法应包括对边按权值递增排序，上述算法假设边已排序的情况下，时间复杂度为 $O(n^2)$ 。

如果给定的带权连通无向图 G 有 e 条边, n 个顶点，采用堆排序 (在第 11 章中介绍) 对边按权值递增排序，那么用克鲁斯卡尔算法构造最小生成树的时间复杂度降为 $O(e \log_2 e)$ 。由于它与 n 无关，只与 e 有关，所以说克鲁斯卡尔算法适合于稀疏图。

6.5 最短路径

6.5.1 路径的概念

在一个无权的图中，若从一顶点到另一顶点存在着一一条路径，则称该路径长度为该路径上所经过的边的数目，它等于该路径上的顶点数减1。

由于从一顶点到另一顶点可能存在着多条路径，每条路径上所经过的边数可能不同，即路径长度不同，我们把路径长度最短（即经过的边数最少）的那条路径叫做最短路径，其路径长度叫做最短路径长度或最短距离。

对于带权的图，考虑路径上各边上的权值，则通常把一条路径上所经边的权值之和定义为该路径的路径长度或称带权路径长度。

从源点到终点可能不止一条路径，把带权路径长度最短的那条路径称为最短路径，其路径长度（权值之和）称为最短路径长度或者最短距离。

6.5.2 从一个顶点到其余各顶点的最短路径

问题：给定一个带权有向图 G 与源点 v ，求从 v 到 G 中其他顶点的最短路径，并限定各边上的权值大于或等于 0。

采用狄克斯特拉 (Dijkstra) 算法求解

基本思想是：设 $G=(V,E)$ 是一个带权有向图，把图中顶点集合 V 分成两组：

第一组为已求出最短路径的顶点集合（用 S 表示，初始时 S 中只有一个源点，以后每求得一条最短路径 $v, \dots, v_k$ ，就将 v_k 加入到集合 S 中，直到全部顶点都加入到 S 中，算法就结束了）

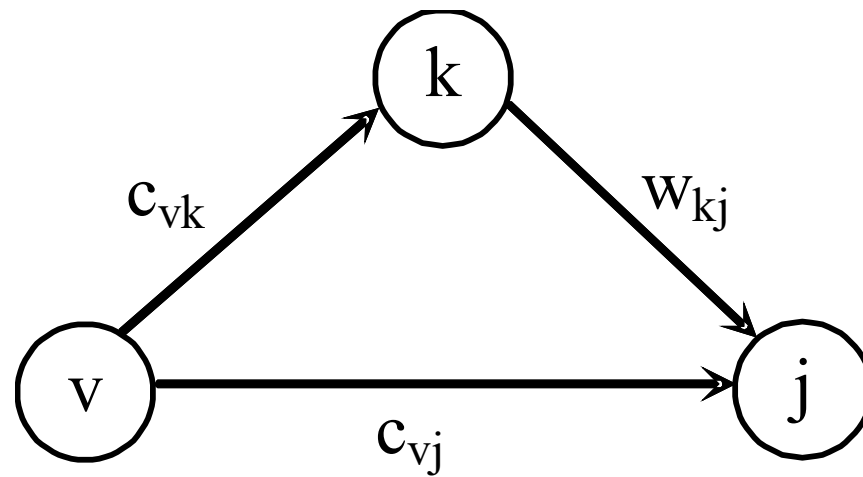
第二组为其余未确定最短路径的顶点集合（用 U 表示）。

按最短路径长度的递增次序依次把第二组的顶点加入 S 中。在加入的过程中，总保持从源点 v 到 S 中各顶点的最短路径长度不大于从源点 v 到 U 中任何顶点的最短路径长度。

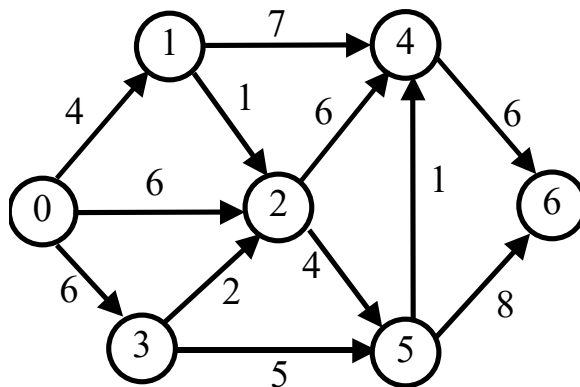
此外，每个顶点对应一个距离， S 中的顶点的距离就是从 v 到此顶点的最短路径长度， U 中的顶点的距离从 v 到此顶点只包括 S 中的顶点为中间顶点的当前最短路径长度。

狄克斯特拉算法的具体步骤如下：

- (1) 初始时, S 只包含源点, 即 $S=\{v\}$, v 的距离为 0。
 U 包含除 v 外的其他顶点, U 中顶点 u 距离为边上的权 (若 v 与 u 有边 $\langle v, u \rangle$) 或 ∞ (若 u 不是 v 的出边邻接点)。
- (2) 从 U 中选取一个距离 v 最小的顶点 k , 把 k 加入 S 中 (该选定的距离就是 v 到 k 的最短路径长度)。
- (3) 以 k 为新考虑的中间点, 修改 U 中各顶点的距离: 若从源点 v 到顶点 $u(u \in U)$ 的距离 (经过顶点 k) 比原来距离 (不经过顶点 k) 短, 则修改顶点 u 的距离值, 修改后的距离值的顶点 k 的距离加上边 $\langle k, u \rangle$ 上的权。
- (4) 重复步骤 (2) 和 (3) 直到所有顶点都包含在 S 中。



V 到 j 的最小距离 = $\text{MIN}(c_{vk} + w_{kj}, c_{vj})$



S

U

dist[]

path[]

0 1 2 3 4 5 6

0 1 2 3 4 5 6

{0}	{1, 2, 3, 4, 5, 6}	{0, 4, 6, 6, ∞ , ∞ , ∞ }	{0, 0, 0, 0, -1, -1, -1}
{0, <u>1</u> }	{2, 3, 4, 5, 6}	{0, 4, 5 , 6, 11 , ∞ , ∞ }	{0, 0, 1 , 0, 1 , -1, -1}
{0, 1, <u>2</u> }	{3, 4, 5, 6}	{0, 4, 5, 6, 11, 6 , ∞ }	{0, 0, 1, 0, 1, 2 , -1}
{0, 1, 2, <u>3</u> }	{4, 5, 6}	{0, 4, 5, 6, 11, 6, ∞ }	{0, 0, 1, 0, 1, 2, -1}
{0, 1, 2, 3, <u>5</u> }	{4, 6}	{0, 4, 5, 6, 10 , 6, 17 }	{0, 0, 1, 0, 5 , 2, 5 }
{0, 1, 2, 3, 5, <u>4</u> }	{6}	{0, 4, 5, 6, 10, 6, 16 }	{0, 0, 1, 0, 5, 2, 4 }
{0, 1, 2, 3, 5, 4, <u>6</u> }	{}	{0, 4, 5, 6, 10, 6, 16}	{0, 0, 1, 0, 5, 2, 4}

狄克斯特拉算法如下 (n 为图 G 的顶点数, v_0 为源点编号) :

```
void Dijkstra(int cost[][MAXV],int n,int v0)
```

```
{   int dist[MAXV],path[MAXV]; int s[MAXV];int mindis,i,j,u;
```

```
    for (i=0;i<n;i++)
```

```
    {   dist[i]=cost[v0][i];           /* 距离初始化 */
```

```
        s[i]=0;                       /*s[] 置空 */
```

```
        if (cost[v0][i]<INF)          /* 路径初始化 */
```

```
            path[i]=v0;
```

```
        else
```

```
            path[i]=-1;
```

```
    }
```

```
    s[v0]=1;path[v0]=0;  /* 源点编号 v0 放入 s 中 */
```

```

for (i=0;i<n;i++) /* 循环直到所有顶点的最短路径都求出 */
{   mindis=INF;
    u=-1;
    for (j=0;j<n;j++) /* 选取不在 s 中且具有最小距离的顶点
u*/
        if (s[j]==0 && dist[j]<mindis)
            { u=j; mindis=dist[j]; }
    s[u]=1; /* 顶点 u 加入 s 中 */
    for (j=0;j<n;j++) /* 修改不在 s 中的顶点的距离 */
        if (s[j]==0)
            if (cost[u][j]<INF && dist[u]+cost[u][j]<dist[j])
                { dist[j]=dist[u]+cost[u][j]; path[j]=u; }
}
Dispath(dist,path,s,n,v0); /* 输出最短路径 */
}

```



```
void Ppath(int path[],int i,int v0)
```

```
/* 前向递归查找路径上的顶点 */
```

```
{ int k;
```

```
    k=path[i];
```

```
    if (k==v0) return;
```

```
    Ppath(path,k,v0);
```

```
    printf("%d,",k);
```

```
}
```

```
/* 找到了起点则返回 */
```

```
/* 找 k 顶点的前一个顶点 */
```

```
/* 输出 k 顶点 */
```

```
void Dispath(int dist[],int path[],int s[],int n,int v0)
```

```
{   int i;
```

```
    for (i=0;i<n;i++)
```

```
        if (s[i]==1)
```

```
        {   printf(" 从 %d 到 %d 的最短路径长度为 :
```

```
                %d\t 路径为 :",v0,i,dist[i]);
```

```
                printf("%d,",v0);           /* 输出路径上的起点 */
```

```
                Ppath(path,i,v0);           /* 输出路径上的中间点 */
```

```
                printf("%d\n",i);           /* 输出路径上的终点 */
```

```
        }
```

```
        else printf(" 从 %d 到 %d 不存在路径 \n",v0,i);
```

```
}
```

6.5.3 每对顶点之间的最短路径

问题：对于一个各边权值均大于零的有向图，对每一对顶点 $v_i \neq v_j$ ，求出 v_i 与 v_j 之间的最短路径和最短路径长度。

可以通过以每个顶点作为源点循环求出每对顶点之间的最短路径。除此之外，弗洛伊德 (Floyd) 算法也可用于求两顶点之间最短路径。

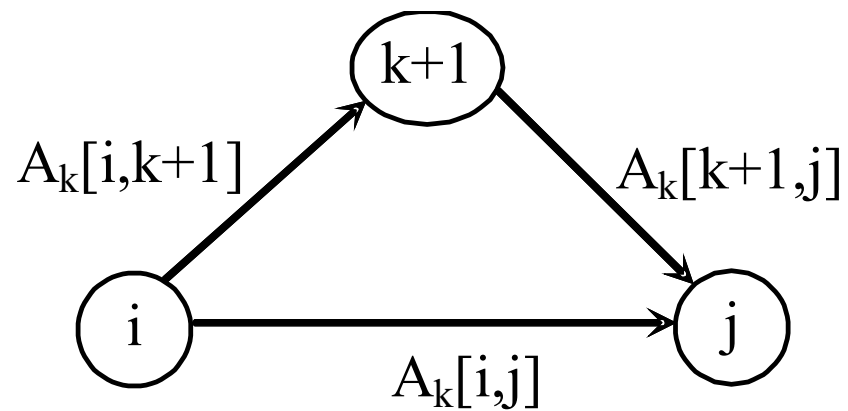
假设有向图 $G=(V,E)$ 采用邻接矩阵 cost 存储，另外设置一个二维数组 A 用于存放当前顶点之间的最短路径长度，分量 $A[i][j]$ 表示当前顶点 v_i 到顶点 v_j 的最短路径长度。

弗洛伊德算法的基本思想是递推产生一个矩阵序列 $A_0, A_1, \dots, A_k, \dots, A_n$ ，其中 $A_k[i][j]$ 表示从顶点 v_i 到顶点 v_j 的路径上所经过的顶点编号不大于 k 的最短路径长度。

初始时, 有 $A_{-1}[i][j]=\text{cost}[i][j]$ 。当求从顶点 v_i 到顶点 v_j 的路径上所经过的顶点编号不大于 $k+1$ 的最短路径长度时, 要分两种情况考虑:

一种情况是该路径不经过顶点编号为 $k+1$ 的顶点, 此时该路径长度与从顶点 v_i 到顶点 v_j 的路径上所经过的顶点编号不大于 k 的最短路径长度相同;

另一种情况是从顶点 v_i 到顶点 v_j 的最短路径上经过编号为 $k+1$ 的顶点。



$$A_{k+1}[i, j] = \text{MIN}(A_k[i, j], A_k[i, k+1] + A_k[k+1, j])$$

那么，该路径可分为两段：

- (1) 从顶点 v_i 到顶点 v_{k+1} 的最短路径；
- (2) 从顶点 v_{k+1} 到顶点 v_j 的最短路径。

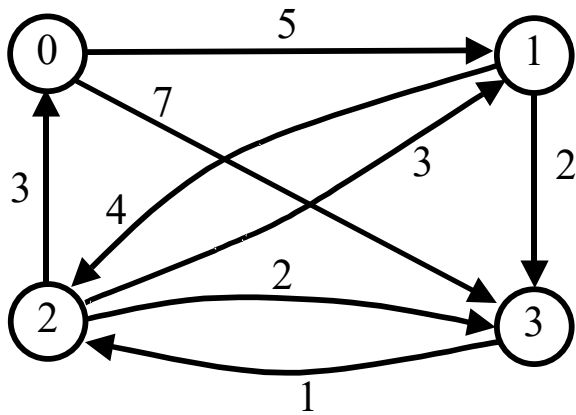
此时最短路径长度等于这两段路径长度之和。这两种情况中的较小值，就是所要求的从顶点 v_i 到顶点 v_j 的路径上所经过的顶点编号不大于 $k+1$ 的最短路径。

弗洛伊德思想可用如下的表达式来描述：

$$A_{-1}[i][j] = \text{cost}[i][j]$$

$$A_{k+1}[i][j] = \text{MIN} \{ A_k[i][j], A_k[i][k+1] + A_k[k+1][j] \}$$

$$(0 \leq k \leq n-1)$$



$$\begin{bmatrix} 0 & 5 & \infty & 7 \\ \infty & 0 & 4 & 2 \\ 3 & 3 & 0 & 2 \\ \infty & \infty & 1 & 0 \end{bmatrix}$$

$$A_{-1} = \begin{bmatrix} 0 & 5 & \infty & 7 \\ \infty & 0 & 4 & 2 \\ 3 & 3 & 0 & 2 \\ \infty & \infty & 1 & 0 \end{bmatrix}$$

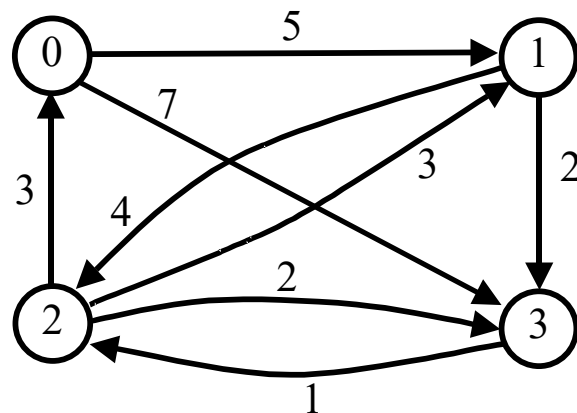
$$Path_{-1} = \begin{bmatrix} -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 \end{bmatrix}$$

采用弗洛伊德算法求解过程

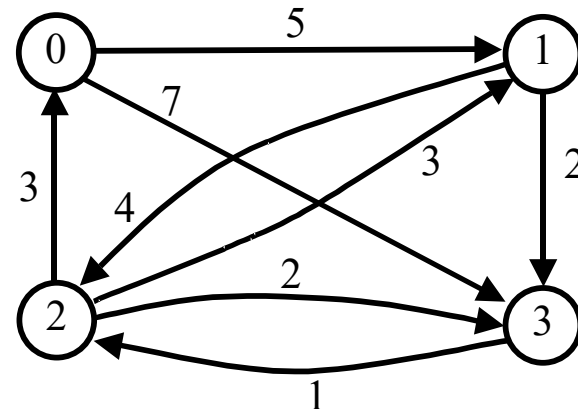
考虑顶点 v_0 , $A_0[i][j]$ 表示由 v_i 到 v_j 经由顶点 v_0 的最短路径。只有从 v_2 到 v_1 经过 v_0 的路径和从 v_2 到 v_3 经过 v_0 的路径, 不影响 v_2 到 v_1 和 v_2 到 v_3 的路径长度, 因此, 有:

$$A_0 = \begin{bmatrix} 0 & 5 & \infty & 7 \\ \infty & 0 & 4 & 2 \\ 3 & 3 & 0 & 2 \\ \infty & \infty & 1 & 0 \end{bmatrix}$$

$$Path_0 = \begin{bmatrix} -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 \end{bmatrix}$$



考虑顶点 v_1 , $A_1[i][j]$ 表示由 v_i 到 v_j , 经由顶点 v_1 的最短路径。
 存在路径 $v_0-v_1-v_2$, 路径长度为 6, 将 $A[0][2]$ 改为 6, $path[0][2]$ 改为 1, 因此, 有:



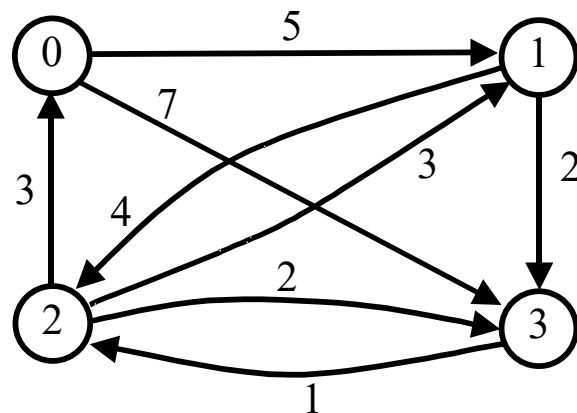
$$A_1 = \begin{bmatrix} 0 & 5 & 6 & 7 \\ \infty & 0 & 4 & 2 \\ 3 & 3 & 0 & 2 \\ \infty & \infty & 1 & 0 \end{bmatrix}$$

$$Path_1 = \begin{bmatrix} -1 & -1 & 1 & -1 \\ -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 \end{bmatrix}$$

考虑顶点 v_2 , $A_2[i][j]$ 表示由 v_i 到 v_j , 经由顶点 v_2 的最短路径。存在路径 $v_3-v_2-v_0$, 长度为 4, 将 $A[3][0]$ 改为 4; 存在路径 $v_3-v_2-v_1$, 长度为 4, 将 $A[3][1]$ 改为 4。存在路径 $v_1-v_2-v_0$, 长度为 7, 将 $A[1][0]$ 改为 7。将 $path[3][0]$ 、 $path[3][1]$ 和 $path[1][0]$ 均改为 2。因此, 有:

$$A_2 = \begin{bmatrix} 0 & 5 & 9 & 7 \\ 7 & 0 & 4 & 2 \\ 3 & 3 & 0 & 2 \\ 4 & 4 & 1 & 0 \end{bmatrix}$$

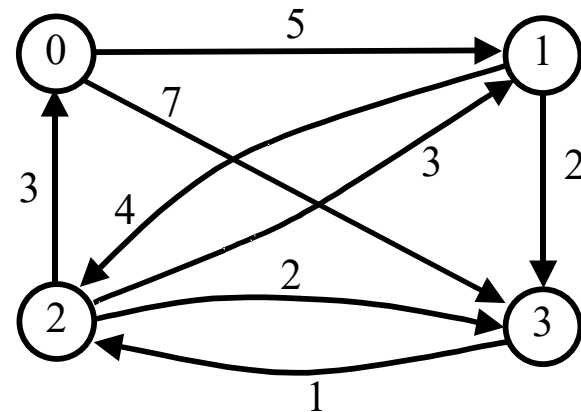
$$Path_2 = \begin{bmatrix} -1 & -1 & 1 & -1 \\ 2 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 \\ 2 & 2 & -1 & -1 \end{bmatrix}$$



考虑顶点 v_3 , $A_3[i][j]$ 表示由 v_i 到 v_j 经由顶点 v_3 的最短路径。存在路径 $v_0-v_3-v_2$, 长度为 8 比原长度短, 将 $A[0][2]$ 改为 8; 存在路径 $v_1-v_3-v_2-v_0$, 长度为 6 ($A[1][3]+A[3][0]=2+4=6$) 比原长度短, 将 $A[1][0]$ 改为 6; 存在路径 $v_1-v_3-v_2$, 长度为 3, 比原长度短, 将 $A[1][2]$ 改为 3; 将 $path[0][2]$ 、 $path[1][0]$ 后 $path[1][2]$ 均改为 3。因此, 有:

$$A_3 = \begin{bmatrix} 0 & 5 & 8 & 7 \\ 6 & 0 & 3 & 2 \\ 3 & 3 & 0 & 2 \\ 4 & 4 & 1 & 0 \end{bmatrix}$$

$$Path_3 = \begin{bmatrix} -1 & -1 & 3 & -1 \\ 3 & -1 & 3 & -1 \\ -1 & -1 & -1 & -1 \\ 2 & 2 & -1 & -1 \end{bmatrix}$$



A=

$$\begin{bmatrix} 0 & 5 & 8 & 7 \\ 6 & 0 & 3 & 2 \\ 3 & 3 & 0 & 2 \\ 4 & 4 & 1 & 0 \end{bmatrix}$$

Path=

$$\begin{bmatrix} -1 & -1 & 3 & -1 \\ 3 & -1 & 3 & -1 \\ -1 & -1 & -1 & -1 \\ 2 & 2 & -1 & -1 \end{bmatrix}$$

从 0 到 0 路径为: 0,0

路径长度为: 0

从 0 到 1 路径为: 0,1

路径长度为: 5

从 0 到 2 路径为: 0,3,2

路径长度为: 8

从 0 到 3 路径为: 0,3

路径长度为: 7

从 1 到 0 路径为: 1,3,2,0

路径长度为: 6

从 1 到 1 路径为: 1,1

路径长度为: 0

从 1 到 2 路径为: 1,3,2

路径长度为: 3

从 1 到 3 路径为: 1,3

路径长度为: 2

从 2 到 0 路径为: 2,0

路径长度为: 3

从 2 到 1 路径为: 2,1

路径长度为: 3

从 2 到 2 路径为: 2,2

路径长度为: 0

从 2 到 3 路径为: 2,3

路径长度为: 2

从 3 到 0 路径为: 3,2,0

路径长度为: 4

从 3 到 1 路径为: 3,2,1

路径长度为: 4

从 3 到 2 路径为: 3,2

路径长度为: 1

从 3 到 3 路径为: 3,3

路径长度为: 0

弗洛伊德算法如下：

```
void Floyd(int cost[][MAXV],int n)
{   int A[MAXV][MAXV],path[MAXV][MAXV];int i,j,k;
    for (i=0;i<n;i++)
        for (j=0;j<n;j++)
            { A[i][j]=cost[i][j]; path[i][j]=-1; }
    for (k=0;k<n;k++)
        for (i=0;i<n;i++)
            for (j=0;j<n;j++)
                if (A[i][j]>(A[i][k]+A[k][j]))
                    { A[i][j]=A[i][k]+A[k][j]; path[i][j]=k; }
    Dispath(A,path,n); /* 输出最短路径 */
}
```


6.6 拓扑排序

设 $G=(V,E)$ 是一个具有 n 个顶点的有向图, V 中顶点序列 $v_1, v_2, \dots, v_n$ 称为一个拓扑序列, 当且仅当该顶点序列满足下列条件:

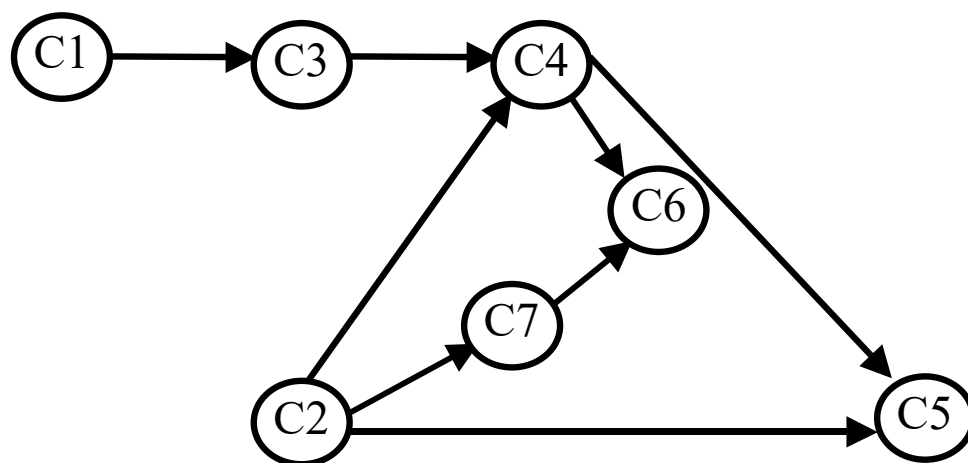
若 $\langle v_i, v_j \rangle$ 是图中的边 (即从顶点 v_i 到 v_j 有一条路径), 则在序列中顶点 v_i 必须排在顶点 v_j 之前。

在一个有向图中找一个拓扑序列的过程称为拓扑排序。

例如，计算机专业的学生必须完成一系列规定的基础课和专业课才能毕业，假设这些课程的名称与相应代号有如下关系：

课程代号	课程名称	先修课程
C1	高等数学	无
C2	程序设计	无
C3	离散数学	C1
C4	数据结构	C2,C3
C5	编译原理	C2,C4
C6	操作系统	C4,C7
C7	计算机组成原理	C2

课程之间的先后关系可用有向图表示：



对这个有向图进行拓扑排序可得到一个拓扑序列： **C1-C3-C2-C4-C7-C6-C5** 。也可得到另一个拓扑序列： **C2-C7-C1-C3-C4-C5-C6**, 还可以得到其他的拓扑序列。学生按照任何一个拓扑序列都可以顺序地进行课程学习。

拓扑排序方法如下：

- (1) 从有向图中选择一个没有前驱 (即入度为 0) 的顶点并且输出它。
- (2) 从网中删去该顶点, 并且删去从该顶点发出的全部有向边。
- (3) 重复上述两步, 直到剩余的网中不再存在没有前驱的顶点为止。

为了实现拓扑排序的算法，对于给定的有向图，采用邻接表作为存储结构，为每个顶点设立一个链表，每个链表有一个表头结点，这些表头结点构成一个数组，表头结点中增加一个存放顶点入度的域 **count**。即将邻接表定义中的 **VNode** 类型修改如下：

```
typedef struct                                /* 表头结点类型 */
{
    Vertex data;                             /* 顶点信息 */
    int count;                                /* 存放顶点入度 */
    ArcNode *firstarc; /* 指向第一条弧 */
} VNode;
```

```

void TopSort(VNode adj[],int n)
{   int i,j;int St[MAXV],top=-1; /* 栈 St 的指针为 top*/
    ArcNode *p;
    for (i=0;i<n;i++)
        if (adj[i].count==0) /* 入度为 0 的顶点入栈 */
        {   top++; St[top]=i;   }
    while (top>-1) /* 栈不为空时循环 */
    {   i=St[top];top--;          /* 出栈 */
        printf("%d ",i); p=adj[i].firstarc;
        while (p!=NULL)
        {   j=p->adjvex; adj[j].count--;
            if (adj[j].count==0)
            {   top++; St[top]=j; }
            p=p->nextarc;          /* 找下一个相邻顶点 */
        }
    }
}

```

6.7 AOE 网与关键路径

若用前面介绍过的带权有向图 (DAG) 描述工程的预计进度, 以顶点表示事件, 有向边表示活动, 边 e 的权 $c(e)$ 表示完成活动 e 所需的时间 (比如天数), 或者说活动 e 持续时间。

图中入度为 0 的顶点表示工程的开始事件 (如开工仪式), 出度为 0 的顶点表示工程结束事件。则称这样的有向图为 AOE 网 (Activity On Edge)。

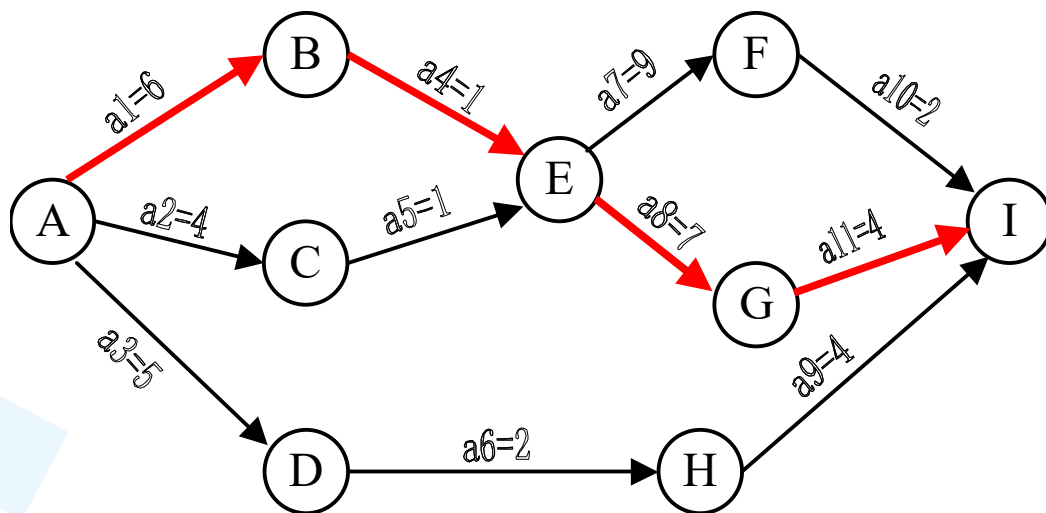
通常每个工程都只有一个开始事件和一个结束事件，因此表示工程的 AOE 网都只有一个入度为 0 的顶点，称为源点 (source)，和一个出度为 0 的顶点，称为汇点 (converge)。

如果图中存在多个入度为 0 的顶点，只要加一个虚拟源点，使这个虚拟源点到原来所有入度为 0 的点都有一条长度为 0 的边，变成只有一个源点。对存在多个出度为 0 的顶点的情况作类似的处理。所以只需讨论单源点和单汇点的情况。

在 AOE 网中，从源点到汇点的所有路径中，具有最大路径长度的路径称为关键路径。完成整个工程的最短时间就是网中关键路径的长度，也就是网中关键路径上各活动持续时间的总和，我们把关键路径上的活动称为关键活动。因此，只要找出 AOE 网中的关键活动，也就找到了关键路径。

注意：在一个 AOE 网中，可以有不止一条的关键路径。

例如，下图表示某工程的 AOE 网。共有 6 个事件和 11 项活动。其中 A 表示开始事件，G 表示结束事件。



几个定义:

(1) 事件 v 的最早可发生时间: 假设事件 x 是源点, 事件 y 为汇点, 并规定事件 x 的发生时间为 0。定义图中任一事件 v 的最早 (event early) 可发生时间 $ve(v)$ 等于 x 到 v 所有路径长度的最大值, 即:

$$ve(v) = \underset{p}{\text{MAX}} \{c(p)\}$$

上式中的 MAX 是对 x 到 v 的所有路径 p 取最大值, $c(p)$ 表示路径 p 的长度 (路径上边长之和), 即:

$$c(p) = \sum_{e \in p} \{c(e)\}$$

完成整个工程所需的最少时间, 等于事件 y 的最早可发生时间 $ve(y)$ 。

(2) 事件 v 的最迟可发生时间：定义在不影响整个工程进度的前提下，事件 v 必须发生的时间称为 v 的最迟 (event late) 可发生时间，记作 $vl(v)$ 。 $vl(v)$ 应等于 $ve(y)$ 与 v 到 y 的最长路径长度之差 (y 是汇点)，即：

$$vl(v) = ve(y) - \max_p \{c(p)\}$$

(3) 关键活动：若活动 v 满足 $ve(v) + c(a_i) = vl(w)$ ，则称活动 a_i 为关键活动。

对关键活动来说，不存在富余时间。显然，关键路径上的活动都是关键活动。找出关键活动的意义在于，可以适当地增加对关键活动的投资（人力、物力等），相应地减少对非关键活动的投资，从而减少关键活动的持续时间，缩短整个工程的工期。

只要计算出各项点的 $ve(v)$ 和 $vl(v)$ 的值, 根据 6.3 等式就能找出所有的关键活动。为了便于计算, 引入下面两个递推式, 其中 x 和 y 分别是源点和汇点。

$$ve(x)=0 \quad \text{MAX}_{\text{所有存在} \langle v, w \rangle \text{ 边的 } v} \{ve(v) + c(\langle v, w \rangle)\}$$

$$ve(w)= \quad w \neq x$$

上式中的 MAX 对所有射入 w 的边 $\langle v, w \rangle$ 取最大值。

$$vl(y)=0 \quad \text{MIN}_{\text{所有存在} \langle v, w \rangle \text{ 边的 } w} \{vl(w) - c(\langle v, w \rangle)\}$$

$$vl(v)= \quad v \neq y$$

求 AOE 网的关键活动的步骤:

(1) 对于源点 x , 置 $ve(x)=0$;

(2) 对 AOE 网进行拓扑排序。如发现回路, 工程无法进行, 则退出; 否则继续下一步。

(3) 按顶点的拓扑次序, 反复用式 6.4, 依次求其余各项点 v 的 $ve(v)$ 值。(实际上, 步骤 (2) 和步骤 (3) 可以合在一起完成, 即一边对顶点进行拓扑排序, 一边求出各点的 $e(v)$ 之值。)

(4) 对于汇点 y , 置 $vl(y)=ve(y)$;

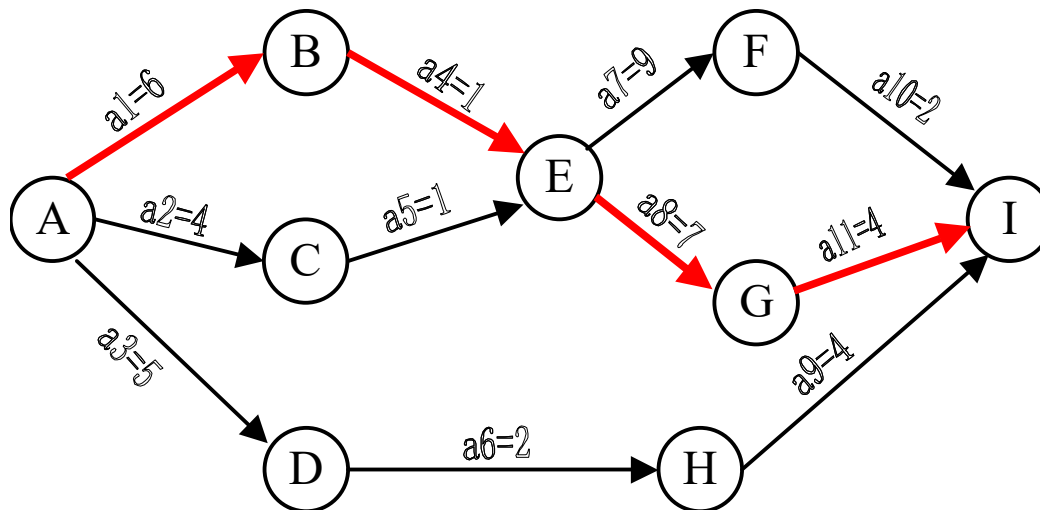
(5) 按顶点拓扑次序之逆序, 反复用式 6.5, 依次求其余各项点 v 的 $vl(v)$ 的值。即对 v 射出的所有边 $\langle v, w \rangle$, 检查是否满足式 6.3, 若是, 则输出该边的有关信息, 指明该边所对应的活动是关键活动。

(6) 活动 a_i 的最早开始时间 $e(i)$ 是该活动的起点所表示的事件最早发生时间。如果由边 $\langle v_j, v_k \rangle$ 表示活动 a_i , 则有 $e(i) = ve(j)$ 。

(7) 活动 a_i 的最迟开始时间 $l(i)$ 是该活动的终点所表示的事件最迟发生时间与该活动的所需时间之差。如果用边 $\langle v_j, v_k \rangle$ 表示活动 a_i , 则有 $l(i) = vl(k) - a_i$ 所需时间。

(8) 一个活动 a_i 的最迟开始时间 $l(i)$ 和其最早开始时间 $e(i)$ 的差额 $d(i)=l(i)-e(i)$ 是该活动完成的时间余量。它是在不增加完成整个工程所需的总时间的情况下, 活动 a_i 可以拖延的时间。

(6) 当一活动的时间余量为零时, 说明该活动必须如期完成, 否则就会拖延完成整个工程的进度。所以我们称 $l(i)-e(i)=0$, 即 $l(i)=e(i)$ 的活动 a_i 是关键活动。



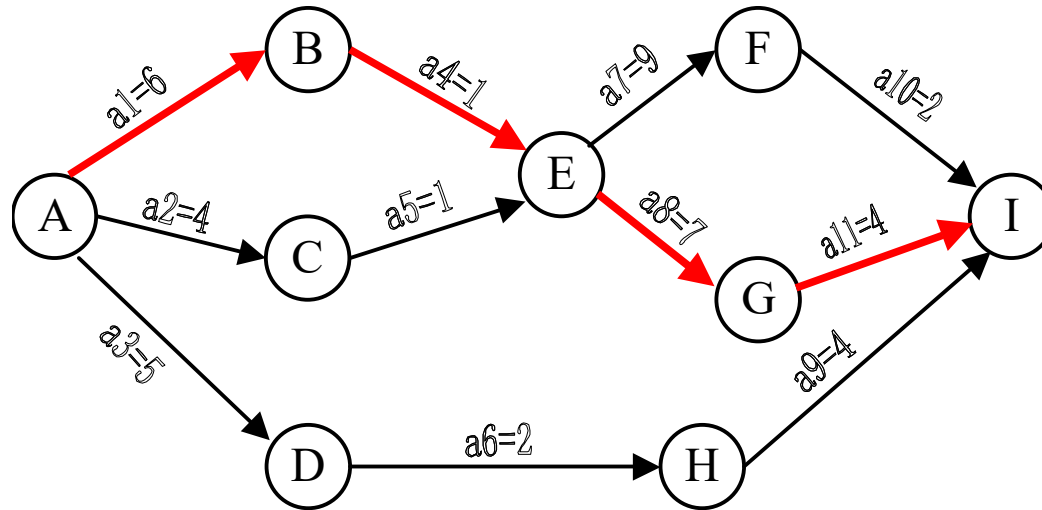
计算各事件的 $e(v)$ 如下:

$$ve(A)=0, ve(B)=ve(A)+c(a_1)=6$$

$$ve(C)=ve(A)+c(a_2)=4$$

$$ve(D)=ve(A)+c(a_3)=5$$

$$ve(E)=\max(ve(B)+c(a_4), ve(C)+c(a_5))=\max\{7, 5\}=7$$



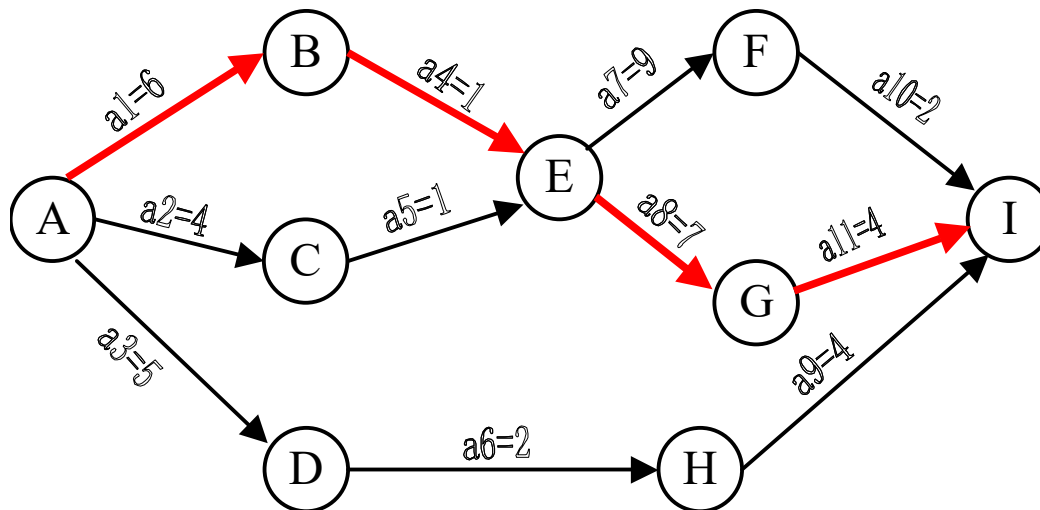
$$ve(F) = ve(E) + c(a7) = 16$$

$$ve(G) = ve(E) + c(a8) = 14$$

$$ve(H) = ve(D) + c(a6) = 7$$

$$ve(I) = \max\{ve(F) + c(a10), ve(G) + c(a11), ve(H) + c(a9)\}$$

$$= \max(18, 18, 11) = 18$$



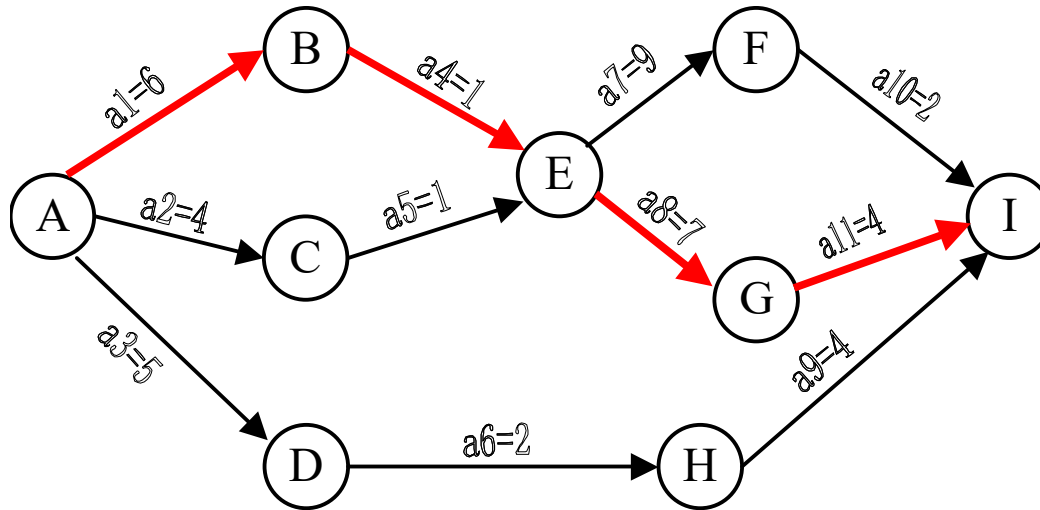
计算各事件的 $vl(v)$ 如下:

$$vl(I)=ve(I)=18$$

$$vl(F)=vl(I)-c(a_{10})=16$$

$$vl(G)=vl(I)-c(a_{11})=14$$

$$vl(H)=vl(I)-c(a_6)=14$$



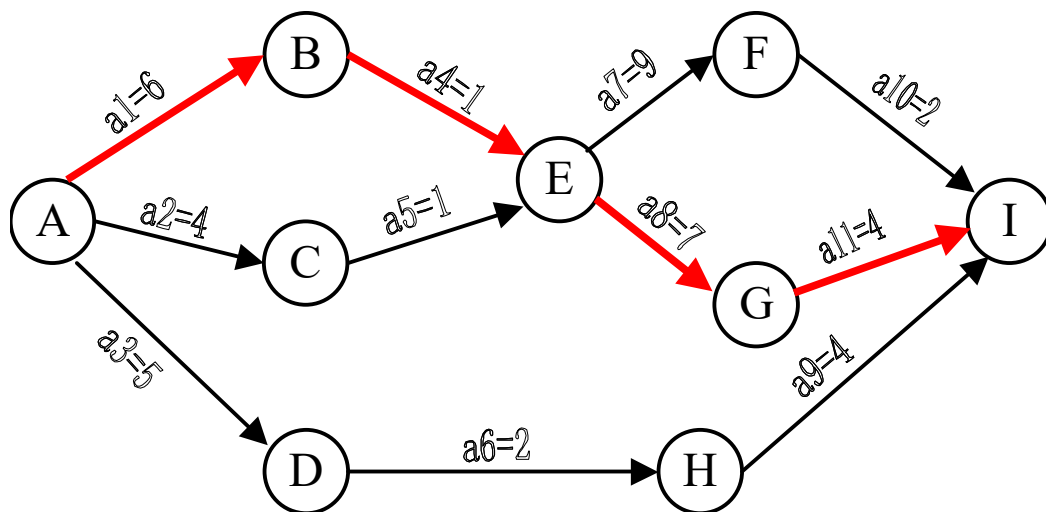
$$vl(E) = \min(vl(F) - c(a7), vl(G) - c(a8)) = \{7, 7\} = 7$$

$$vl(D) = vl(H) - c(a6) = 12$$

$$vl(C) = vl(E) - c(a5) = 6$$

$$vl(B) = vl(E) - c(a4) = 6$$

$$vl(A) = \min(vl(B) - c(a1), vl(C) - c(a2), vl(D) - c(a3)) = \{0, 2, 7\} = 0$$



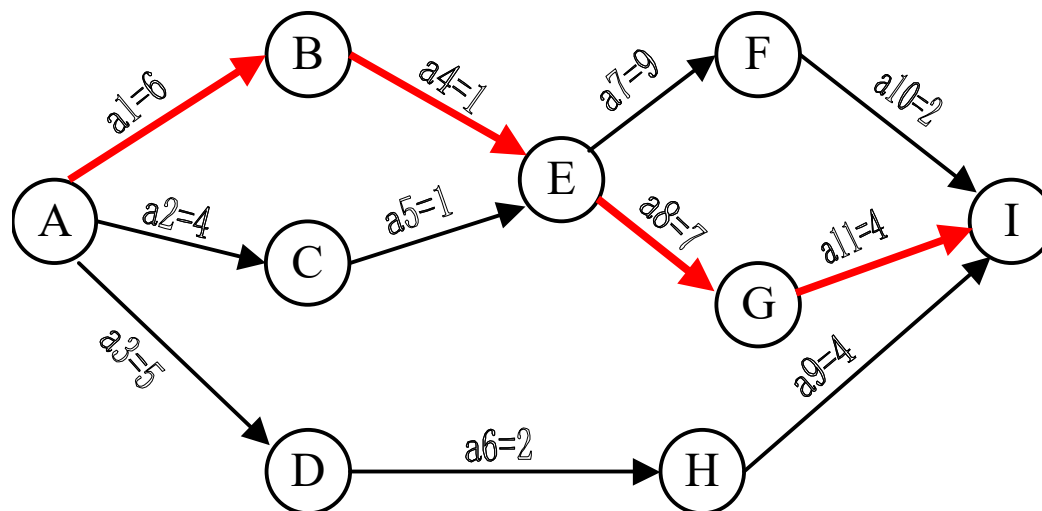
计算各活动的 $e(a)$ 、 $l(a)$ 和 $d(a)$ 如下：

活 动 a_1 :
 $e(1)=ve(A)=0, l(1)=vl(B)-6=0, d(1)=0$

活 动 a_2 :
 $e(2)=ve(A)=0, l(2)=vl(C)-4=2, d(2)=2$

活 动 a_3 :
 $e(3)=ve(A)=0, l(3)=vl(D)-5=7, d(3)=7$

活 动 a_4 :
 $e(4)=ve(B)=6, l(4)=vl(E)-1=6, d(4)=0$



活动 a6 : $e(6)=ve(D)=5, l(6)=vl(H)-2=12, d(6)=7$

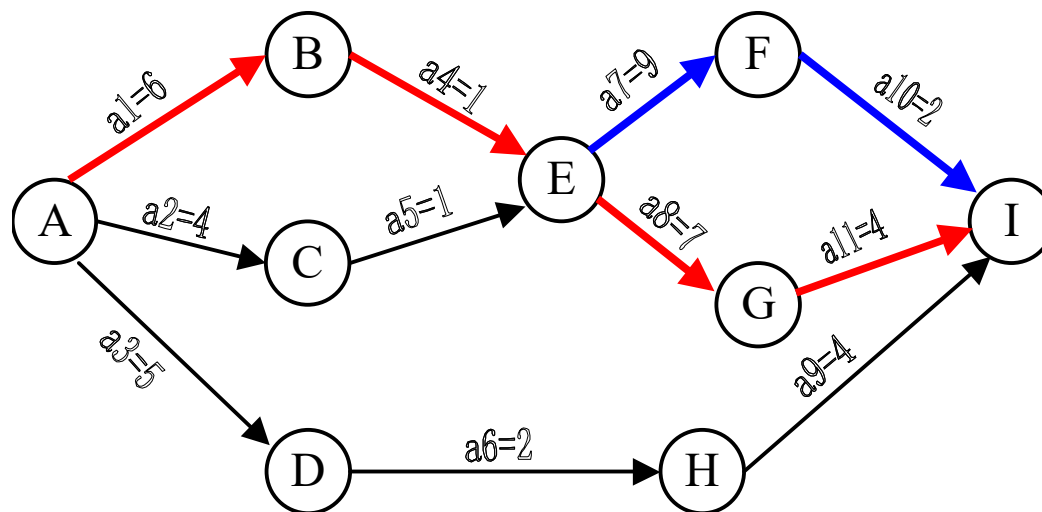
活动 a7 : $e(7)=ve(E)=7, l(7)=vl(F)-6=7, d(7)=0$

活动 a8 : $e(8)=ve(E)=7, l(8)=vl(G)-7=7, d(8)=0$

活动 a6 : $e(6)=ve(H)=7, l(6)=vl(G)-4=10, d(6)=3$

活动 a10 : $e(10)=ve(F)=16, l(10)=vl(I)-2=16, d(10)=0$

活动 a11 : $e(11)=ve(G)=14, l(11)=vl(I)-4=14, d(11)=0$



由此可知,关键活动有 a_{11} 、 a_{10} 、 a_8 、 a_7 、 a_4 、 a_1 , 因此关键路径有两条: A-B-E-F-I 和 A-B-E-G-I。

本章小结

本章基本学习要点如下：

- (1) 掌握图的相关概念，包括图、有向图、无向图、完全图、子图、连通图、度、入度、出度、简单回路和环等定义。
- (2) 重点掌握图的各种存储结构，包括邻接矩阵和邻接表等。
- (3) 重点掌握图的基本运算，包括创建图、输出图、深度优先遍历、广度优先遍历算法等。

(4) 掌握图的其他运算，包括最小生成树、最短路径、拓扑排序等算法。

(5) 灵活运用图这种数据结构解决一些综合应用问题。

练习

教材中 p217 习题 2、3
和 5。

上机实验题

教材中 p217 题 1、2
和 3。

**感谢亲观看此幻灯片，此课件部分内容来源于网络，
如有侵权请及时联系我们删除，谢谢配合！**